
UNIVERSITATEA SAPIENTIA DIN CLUJ-NAPOCA
FACULTATEA DE ȘTIINȚE TEHNICE ȘI UMANISTE,
SPECIALIZAREA INFORMATICĂ

**PROCESE MODERNE DE
DEZVOLTARE WEB**

PROIECT DE DIPLOMĂ

Coordonator științific:
Prof. habil. dr. ing. Szilágyi László

Absolvent:
Seres Tamás

UNIVERSITATEA „SAPIENTIA” din CLUJ-NAPOCA
Facultatea de Științe Tehnice și Umaniste din Târgu Mureș
Specializarea: **Informatică**

Viza facultății:

LUCRARE DE DIPLOMĂ

Coordonator științific:
Prof. dr. habil Szilágyi László

Candidat: **Seres Tamás**
Anul absolvirii: **2026**

a) Tema lucrării de licență:

Procese moderne de dezvoltare web

b) Problemele principale tratate:

- Introducere literară în istoria aplicațiilor web
- Prezentarea procesului de dezvoltare al unei aplicații web moderne
- Dezvoltarea unei aplicații web pentru jurnal de filme ca exemplu în Next.js

c) Desene obligatorii:

- Diagrame UML privind software-ul realizat
- Schema bloc al aplicației

d) Softuri obligatorii:

- Aplicație web pentru jurnal de filme în framework-ul full stack Next.js

e) Bibliografia recomandată:

- Steve McConnell, Rapid Development: Taming Wild Software Schedules, Microsoft Press 1996
- Next.js docs, <https://nextjs.org/docs>
- Claire Drumond, Agile Manifesto, <https://www.atlassian.com/agile/manifesto>

f) Termene obligatorii de consultații: săptămânal

g) Locul și durata practicii: Universitatea „Sapientia” din Cluj-Napoca,
Facultatea de Științe Tehnice și Umaniste din Târgu Mureș
Primit tema la data de: 2 mai 2025
Termen de predare: 29 iunie 2026

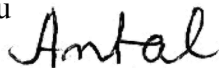
Semnătura Director Departament



Semnătura coordonatorului



Semnătura responsabilului
programului de studiu



Semnătura candidatului



**SAPIENTIA ERDÉLYI MAGYAR
TUDOMÁNYEGYETEM
MAROSVÁSÁRHELYI KAR,
INFORMATIKA SZAK**

**MODERN WEBFEJLESZTÉSI
FOLYAMATOK**

DIPLOMADOLGOZAT

Témavezető:
Dr. habil. Szilágyi László

Végzős hallgató:
Seres Tamás

**SAPIENTIA HUNGARIAN UNIVERSITY OF
TRANSYLVANIA
FACULTY OF TECHNICAL AND HUMAN SCIENCES
INFORMATION TECHNOLOGY SPECIALIZATION**

**PROCESSES OF MODERN WEB
DEVELOPMENT**

BACHELOR THESIS

Scientific advisor:

Dr. habil. Szilágyi László

Student:

Seres Tamás

Declarație

Subsemnatul/a SERES TAMAS, absolvent(ă) al/a programului de studii INFORMATICA, promoția 2026 cunoscând prevederile Legii Educației Naționale 199/2023 și a Codului de etică și deontologie profesională al Universității Sapienția cu privire la furtul intelectual declar pe propria răspundere că prezenta lucrare de licență/proiect de diplomă/disertație se bazează pe activitatea personală, cercetarea/proiectarea este efectuată de mine, informațiile și datele preluate din literatura de specialitate sunt citate în mod corespunzător.

Localitatea, TARGU MURES
Data: 23.06.2026

Absolvent
Semnătura... Seres

Extras

În prezent, aplicațiile web au devenit extrem de răspândite și sunt utilizate zilnic: fie în activitatea profesională, fie pentru comunicare, fie pentru divertisment. Această creștere semnificativă a utilizării aplicațiilor web se datorează faptului că o aplicație accesibilă din browser poate ajunge foarte aproape de nivelul și funcționalitatea programelor desktop.

În spatele aplicațiilor web moderne se află numeroase etape de planificare, decizii, activități de dezvoltare, precum și obstacole tehnice sau chiar financiare, pentru rezolvarea și implementarea cărora este responsabilă o echipă. Fiecare membru are un rol bine definit, de la formularea ideii și proiectarea UI-ului, până la dezvoltare și mentenanță.

Lucrarea își propune să prezinte procesele de planificare ale unei astfel de aplicații web: modul în care se ajunge de la idee la un produs final și ce procese au loc înainte ca dezvoltatorii să scrie chiar și o singură linie de cod sau ca designerii UX să traseze măcar o linie.

Pentru a demonstra acest lucru, este prezentată o echipă de dezvoltare „fictivă”, care proiectează și dezvoltă o aplicație web pentru jurnalizarea filmelor.

Cuvinte cheie: aplicații web, planificare, dezvoltare, design UX, jurnal de filme

Kivonat

Napjainkban rendkívül elterjedté váltak a webes alkalmazások, amelyeket nap mint nap használunk: akár a munkánk során, akár kapcsolattartásra, akár szórakozásra. Ez a hatalmas növekedés a webes alkalmazások használatában annak köszönhető, hogy egy böngészőből használható alkalmazás már-már az asztali programok színvonalát és funkcionalitását is meg tudja közelíteni. A modern webes alkalmazások mögött rengeteg tervezés, döntés, fejlesztés, valamint technikai vagy akár pénzügyi akadály áll, amelyek megoldásáért és kivitelezéséért egy csapat felelős. Mindenkinek megvan a szerepe az ötlet kigondolásától a UI (felhasználói felület) tervezésén át egészen a fejlesztésig és karbantartásig. A dolgozat egy ilyen webes alkalmazás tervezési folyamatait kívánja bemutatni: hogy miként jutunk el magától az ötlettől egy kész termékig, és milyen folyamatok mennek végbe még azelőtt, hogy a fejlesztők leírnának egy sor kódot, vagy a UX-designerek akár egy vonalat is húznának. Ennek demonstrálásához bemutatok egy „fiktív” fejlesztői csapatot, akik egy filmnaplózó weboldalt terveznek és fejlesztenek.

Kulcsszavak: webes alkalmazások, tervezés, fejlesztés, UX-design, filmnaplózó

Abstract

Nowadays, web applications have become extremely widespread and are used on a daily basis, whether for work, communication, or entertainment. This significant growth in the use of web applications is due to the fact that an application accessible from a browser can come very close to the level and functionality of desktop software.

Behind modern web applications there are numerous planning steps, decisions, development tasks, as well as technical or even financial obstacles, whose solution and implementation are the responsibility of a team. Each member has a defined role, from forming the initial idea and designing the UI (user interface) to development and maintenance.

This thesis aims to present the planning processes of such a web application: how an idea can evolve into a finished product, and what processes take place before developers write even a single line of code or UX designers draw even one line.

To demonstrate this, the thesis presents a “fictional” development team that plans and develops a movie logging web application.

Keywords: web applications, planning, development, UX design, movie logging

Tartalomjegyzék

1. Bevezető	1
2. Irodalmi áttekintés	3
2.1. Web alkalmazások kezdetektől napjainkig	3
2.2. Korai világháló	4
2.3. Dinamikus világháló	4
2.4. Fejlett kliensoldali web (AJAX, SPA-k, keretrendszerek)	5
2.5. Mobil és web standardok (reszponzív dizájn)	5
2.6. Jelenlegi állapot és trendek	6
2.7. A web fejlődésének rövid története - idővonal	6
2.8. Modern webfejlesztési alapfogalmak	8
2.9. Webfejlesztési módszertanok	10
2.9.1. Összehasonlítás	10
2.10. Modern webfejlesztői csapat felépítése	11
2.11. Modern webalkalmazások fejlesztési folyamata	13
2.12. A választott technológiai stack áttekintése	14
3. Projektcélok és termékkonceptió	16
3.1. A projekt kiinduló problémája	16
3.2. A PRD szerepe a fejlesztési folyamatban	17
3.3. Termékvízió	19
3.4. Célközönség és perszónák	20

3.5. Piaci kontextus	21
3.6. Termékcélok	22
3.7. Sikerességi metrikák	24
3.8. MVP és verziókra bontás	26
3.9. A user flow-k szerepe	28
3.10. Korlátok és függőségek	30
4. Fejlesztési folyamat	34
4.1. A fejlesztési folyamat kiválasztása	34
4.2. Sprintalapú működés	36
4.3. Feladatkezelés és fejlesztési workflow	37
5. Rendszer specifikációk	40
5.1. Funkcionális követelmények	40
5.2. Nem funkcionális követelmények	43
5.3. Rendszerarchitektúra és technológiai stack	44
5.4. Adatmodellek és fő entitások	45
5.5. API végpontok és adatáramlások	49
5.6. Autentikáció	51
5.7. Külső szolgáltatások és függőségek	52
5.8. Deployment és futtatási környezet	55
6. Megvalósítás	57
6.1. Megvalósítási megközelítés	57
6.2. Autentikáció és védett útvonalak	59
6.3. Filmkeresés és film oldal	60
6.4. Watchlist, watched és rating funkciók	62
6.5. Profiloldal és kliensoldali adatkezelés	64
7. Tesztelés és minőségellenőrzés	66
7.1. A tesztelési stratégia célja	66

7.2. Tesztkörnyezet és használt eszközök	67
7.3. Unit tesztek	68
7.4. Komponens tesztek	69
7.5. CI/CD és automatikus ellenőrzések	70
8. Összefoglaló	72
8.1. Összefoglalás	72
8.2. Továbbfejlesztési lehetőségek	72
Irodalomjegyzék	73

Ábrák jegyzéke

4.1. A WatchDB fejlesztési feladatainak követése GitHub Projects felületen	36
4.2. A WatchDB fejlesztési workflow-ja	38
5.1. A WatchDB fő funkcionális egységei	40
5.2. A filmkeresés fő adatáramlása	42
5.3. A megnézettként jelölés állapotváltozása a filmrészletek oldalon	43
5.4. A WatchDB magas szintű rendszerarchitektúrája	44
5.5. A WatchDB adatbázissémája	46
5.6. A TMDB filmadatok lekérése és normalizálása	48
5.7. A MovieDetails típusdefiníciója	49
5.8. A megnézettként jelölés adatáramlása	51
5.9. A projekt fő függőségei a package.json alapján	54
5.10. A projekt futtatási scriptjei a package.json alapján	55
6.1. A WatchDB projekt mappastruktúrája	58
6.2. Hiba bejelentkezési kísérlet	59
6.3. A filmkeresési találatok megjelenítése	61
6.4. A film oldal felépítése	62
6.5. A megnézettként jelöléshez tartozó modal	63
6.6. A profiloldal felhasználói felülete	64
7.1. Sikertelen CI pipeline a GitHub Actions felületén	67

1. fejezet

Bevezető

A webalkalmazások napjainkban a digitális szolgáltatások egyik legelterjedtebb formáját jelentik. Böngészőből elérhetők, több eszközön használhatók, és gyakran egyszerre kell megfelelniük felhasználói élményre, teljesítményre, biztonságra, skálázhatóságra és karbantarthatóságra vonatkozó elvárásoknak. Egy modern webalkalmazás fejlesztése ezért nem csupán programozási feladat, hanem tervezési, termékfejlesztési és technológiai döntések sorozata is.

A dolgozat célja egy modern webalkalmazás tervezési és fejlesztési folyamatának bemutatása egy konkrét projekt, a WatchDB példáján keresztül. A hangsúly nem kizárólag az elkészült alkalmazáson van, hanem azon a folyamaton is, amely a projektötlettől a követelmények meghatározásán, az MVP (Minimum Viable Product) kijelölésén, a technológiai döntéseken és az implementáción keresztül vezet a működő rendszerig. Ennek megfelelően az alkalmazás a dolgozatban esettanulmányként jelenik meg: segítségével bemutatatható, hogyan válhat egy termékkoncepció strukturált fejlesztési feladattá.

A WatchDB egy filmkövető webalkalmazás koncepciójára épül. A projekt kiinduló problémája, hogy a felhasználók különböző platformokon néznek filmeket, miközben gyakran nincs egységes helyük arra, hogy nyilvántartsák a megnézett filmeket, vezessék a később megtekintendő filmek listáját, illetve értékeljék saját filmélményeiket. A termék hosszabb távú víziója közösségi irányba is bővíthető: a felhasználók követhetik egymás aktivitását, felfedezhetnek ajánlásokat, és megoszthatják filmnézési szokásaikat. A dolgozatban bemutatott fejlesztési fókusz ugyanakkor az első, megvalósítható változat, vagyis az MVP köré szerveződik.

A projekt előkészítésének fontos eleme a Product Requirements Document (PRD), amely a termék

célját, célközönségét, fő funkcióit, sikerességi szempontjait, korlátait és fejlesztési ütemezését foglalja össze. A PRD szerepe ebben a dolgozatban az, hogy láthatóvá tegye: a fejlesztés nem közvetlenül kódolással indul, hanem a probléma, a felhasználói igények és a megvalósítható terjedelem tisztázásával. Ez különösen fontos az MVP meghatározásánál, ahol el kell dönteni, hogy a teljes termékvízióból mely funkciók szükségesek az első használható verzióhoz, és melyek kerülhetnek későbbi fejlesztési szakaszba.

A dolgozat felépítése ezt a gondolatmenetet követi. Az irodalmi áttekintés bemutatja a webalkalmazások fejlődését, a modern webfejlesztés alapfogalmait, a fejlesztési módszertanokat, a webfejlesztői csapat szerepköreit, az SDLC folyamatát és a választott technológiai stack szakmai hátterét. Ezt követően a projektcélok és termékkonceptió fejezete a WatchDB PRD-jéből kiindulva ismerteti a termék célját és tervezési keretét. A további fejezetek a követelmények, architektúrais döntések, gyakorlati megvalósítás és értékelés bemutatásán keresztül tárgyalják, hogyan jelennek meg ezek a döntések egy konkrét webalkalmazás fejlesztése során.

2. fejezet

Irodalmi áttekintés

2.1. Web alkalmazások kezdetektől napjainkig

A világháló (WWW) használatának kezdetét az 1989–1991 közötti periódusra datálják. Kezdeti úttörője Tim Berners-Lee volt, a CERN egyik munkatársa, aki a web alapvető komponensei közé tartozó HTTP-t (HyperText Transfer Protocol), HTML-t (HyperText Markup Language), URL-t, valamint az első webböngészőt és webszerveret is megvalósította [1]. Az ekkor készült weboldalak főként statikusak voltak, majd a szerveroldali programozás és az adatbázisok használata lehetővé tette a dinamikus tartalom előállítását [2]. A 2000-es évektől a kliensoldali interaktivitás az AJAX és a böngészőoldali JavaScript használatával vált egyre hangsúlyosabbá [3]. A mobileszközök elterjedése a responzív dizájnt tette alapvető megközelítéssé [4], a skálázhatóságot pedig a felhőalapú számítástechnika segítette [5].

A modern fejlesztési folyamatok manapság nagy hangsúlyt fektetnek a DevOps szemléletre, a folyamatos integrációra, szállításra és telepítésre (CI/CD), az automatizált tesztelésre, valamint a konténerizációra [6]. Architektúráis oldalon a felhőalapú és mikroszolgáltatás-alapú megoldások terjedése vált meghatározóvá [5, 7].

2.2. Korai világháló

A világháló indulása a CERN-hez kötődik: Tim Berners-Lee 1989 márciusában írta meg az első webes javaslatot, majd 1990 végére megvalósította a HTML, a HTTP és az URL alapjait, valamint az első webszervert és böngésző/szerkesztő programot [1, 8]. Az első weboldal és információs szerver az info.cern.ch címen futott, egy CERN-ben használt NeXT számítógépen [8]. Ezek a korai oldalak főként statikus dokumentumok voltak: a webszerver a tárolt HTML-fájlokat küldte vissza a böngészőnek, dinamikus tartalomgenerálás nélkül [9]. A grafikus böngészők közül az NCSA Mosaic 1993-as megjelenése, majd a Mosaic Communications Corp. (később Netscape) 1994-es megalakulása jelentős szerepet játszott a web népszerűsödésében [8, 10]. Ezzel párhuzamosan a HTML is folyamatosan fejlődött: a HTML 4.0 1997-ben, a HTML5 pedig 2014-ben lett W3C Recommendation, miközben a HTML alapvető szerepe továbbra is a webes dokumentumok szerkezetének leírása maradt [11, 12].

2.3. Dinamikus világháló

1993-ban az NCSA (National Center for Supercomputing Applications) kifejlesztette a CGI-t (Common Gateway Interface), ezzel lehetővé téve, hogy a webszerverek külső programokat futtaszanak (például Perl-szkripteket), és menet közben oldalakat generáljanak [13]. Ez volt az első nagy választóvonal a statikus és a dinamikus weboldalak között. A szerveroldali nyelvek (PHP, Python, ASP, Java) adatbázissal párosítva működtették a korai dinamikus oldalakat (fórumokat, online üzleteket) [2, 14]. Ezeket a szerver által futtatott oldalakat minden egyes kérésnél újra kellett tölteni, ám a 2000-es évek végén az AJAX széles körű ismeretének és használatának köszönhetően az oldalak frissülni tudtak anélkül, hogy teljesen újra kellett volna őket tölteni, így gördülékenyebb felhasználói felületet biztosítottak [3, 15]. Ugyanakkor ezeknek az oldalaknak az alapja továbbra is a kliens és a szerver közötti kérés–válasz modellt követte [16].

2.4. Fejlett kliensoldali web (AJAX, SPA-k, keretrendszerek)

A 2000-es évek közepétől a webalkalmazások egyre inkább „alkalmazásszerűvé” váltak [17]. Az AJAX (a kifejezést 2005-ben vezették be) lehetővé tette, hogy a böngészőben futó JavaScript aszinkron módon kérjen le adatokat [3, 15]. Az olyan keretrendszerek, mint a jQuery (2006), Angular (2010), React (2013), Vue stb., lehetővé tették az egyoldalas alkalmazások (SPA-k) létrehozását, ahol a tartalom dinamikusan töltődik be kliensoldalon [18, 17]. Ezek az alkalmazások gyakran REST vagy GraphQL API-kkal kommunikálnak [19, 20].

A HTML5 (W3C ajánlás, 2014 október) szabványosított új API-kat (Canvas, WebSockets, Web Storage, videó/hang támogatás) a fejlettebb funkciók érdekében [12, 21, 22]. Ennek eredményeként a modern weboldalak képesek összetett logikát futtatni a böngészőben, offline gyorsítótárazást és push értesítéseket használni [23]. A „statikus weboldal” és a „webalkalmazás” közötti különbség elmosódott, mivel még a tartalomközpontú oldalak is átvették a kliensoldali renderelést [24, 17].

2.5. Mobil és web standardok (reszponzív dizájn)

Az okostelefonok korszaka újratervezésre kényszerítette a webalkalmazásokat [25]. Az olyan webes szabványok, mint a CSS3 media query-k és a flexibilis elrendezések, lehetővé tették a reszponzív dizájnt: olyan oldalak létrehozását, amelyek alkalmazkodnak a különböző képernyőméretekhez [26, 27]. A W3C HTML5 és CSS3 specifikációi támogatást nyújtottak mobilbarát funkciókhoz is (pl. érintőképernyő általi események) [12, 28].

Sok weboldal átvette a „mobile-first” tervezési szemléletet [29]. A körülbelül 2015-ben megjelent Progressive Web Appok (PWA-k) Service Workereket és manifest fájlokat használnak annak érdekében, hogy natív alkalmazásokhoz hasonló élményt nyújtsanak (offline mód, telepíthető ikonok) [30, 31]. Ezek a modern HTML5/JavaScript technológiákra és böngésző API-kra (kamera, helymeghatározás stb.) épülnek, hogy fejlett funkcionalitást biztosítsanak [32, 33].

2.6. Jelenlegi állapot és trendek

A mai webes technológiai stackek gyakran mikroszolgáltatásokat (API-kon keresztül kommunikáló, elkülönült szolgáltatásokat) használnak, amelyeket sokszor szerver nélküli környezetben (serverless) futtatnak, például az AWS Lambda szolgáltatásával, amelyet 2014-ben vezettek be [34].

A biztonság és az adatvédelem kiemelten fontossá vált: a HTTPS mára szinte mindenhol elterjedt [35]. A Let's Encrypt automatizálta a tanúsítványok kiadását, és a böngészők egyre inkább mindenhol titkosítást követelnek meg [36, 37]. A tartalomszolgáltató hálózatok (CDN-ek) és az edge computing javítják a teljesítményt [38].

A fejlesztési folyamatok iteratívvá váltak: az Agile/DevOps csapatok gyakori frissítéseket telepítenek, és automatizált tesztelési eszközöket valamint monitorozási megoldásokat használnak [39, 6]. A webes szabványok folyamatosan fejlődnek, miközben a WHATWG és a W3C a HTML, az Ecma International pedig a JavaScript alapját adó ECMAScript továbbfejlesztésén dolgozik [40, 41]. A böngészők új technológiákkal (például WebAssembly) bővítik a platformot [42].

A webes és a natív alkalmazások közötti határ egyre inkább elmosódik, ahogy a PWA-k egyre népszerűbbé válnak [31]. A hibrid keretrendszerek és platformok, mint a React Native vagy a Flutter web, szintén ezt a közeledést erősítik [43, 44].

2.7. A web fejlődésének rövid története - idővonal

1960-as évek

Az internet alapjai

Az ARPANET és a csomagkapcsolt adatátvitel megjelenése megalapozta a későbbi, egymással összekapcsolt számítógépes hálózatokat [45, 46].

1989–1991

A World Wide Web megszületése

Tim Berners-Lee a CERN-ben kidolgozta a World Wide Web koncepcióját, majd megjelentek a web alapvető elemei: a HTML, a HTTP, az URL, az első webszerver és az első böngésző [1, 8].

1990-es évek

A statikus web korszaka

A korai weboldalak főként HTML-alapú, információközlő dokumentumok voltak, amelyeket a webszerverek dinamikus tartalomgenerálás nélkül szolgáltatnak ki [9]. A grafikus böngészők, különösen az NCSA Mosaic, jelentősen hozzájárultak a web elterjedéséhez [10].

2000-es évek

Dinamikus webalkalmazások

A szerveroldali programozás és az adatbázisok használata lehetővé tette a dinamikus tartalom előállítását [2]. Az AJAX elterjedése interaktívabb, részleges frissítésre képes webes felületeket eredményezett [3, 15].

2010-es évek

Modern kliensoldali web

Az egyoldalas alkalmazások és a frontend keretrendszerek a böngészőt egyre inkább alkalmazásfuttatási környezetté tették [18, 17]. Ezzel párhuzamosan a felhőalapú infrastruktúra és a DevOps szemlélet meghatározóvá vált a webalkalmazások fejlesztésében és üzemeltetésében [5, 6].

A PWA-k és a szerver nélküli architektúrák tovább közelítették egymáshoz a webes és natív alkalmazások felhasználói élményét [31, 34]. A mesterséges intelligencia és az automatizáció új fejlesztési és tartalom-előállítási irányokat nyitott a weben [47].

2.8. Modern webfejlesztési alapfogalmak

Webalkalmazásnak nevezzük azt a böngészőn keresztül elérhető szoftvert, amely a felhasználói interakciókra dinamikus válaszokat ad, és gyakran kliensoldali, illetve szerveroldali komponensekből áll [48, 49]. Ezzel szemben a statikus weboldal olyan egyszerű HTML-dokumentum, ahol nincs szerveroldali feldolgozás – így az nem minősül webalkalmazásnak, hiszen a kiszolgáló nem hajt végre dinamikus kódot a kérésekre [9, 2]. A modern webalkalmazások gyakran két csoportba sorolhatók: Multi-Page Application (MPA) és Single-Page Application (SPA) [50]. Az MPA hagyományosan külön oldalakra bontja a tartalmat, minden felhasználói művelet után új oldalt tölt be a szerverről [50]. Ezzel szemben az SPA egyetlen HTML-oldalt használ, ahol a tartalom dinamikusan JavaScript segítségével frissül anélkül, hogy a felhasználót át kellene irányítani egy másik oldalra [18]. Az SPA lényege, hogy a legtöbb számítás a kliensoldalon történik. Az alkalmazás egyszer betölt egy üres HTML sablont, majd a JavaScript kéri le és frissíti dinamikusan a tartalmat [18, 50].

Mindkét fajtának megvan az előnye és a hátránya, az alábbi táblázatban egy összehasonlítást láthatunk a különböző megoldásokról:

Architektúra / renderelés	Módszer	Előnyök	Hátrányok
Statikus weboldal	Előre elkészített HTML-fájlok kiszolgálása, szerveroldali feldolgozás nélkül [9].	Gyors betöltés, egyszerű hostolás, kevés szerveroldali függőség.	Kevés interaktivitás; a tartalom módosítása kézi frissítést vagy újragenerálást igényel.
Többoldalas alkalmazás (MPA)	A felhasználó külön URL-ek között navigál, az új oldalak teljes dokumentumként töltődnek be [50].	Keresőoptimalizálás szempontjából kedvező, hagyományos és széles körben ismert megközelítés.	A navigáció lassabb lehet, mert minden oldalváltás új dokumentum betöltésével jár.
Egyoldalas alkalmazás (SPA)	Az első betöltés után a kliensoldali JavaScript frissíti a felületet, teljes oldalújratöltés nélkül [18].	Alkalmazásszerű felhasználói élmény, gyors belső navigáció, offline támogatás lehetősége.	Nagy JavaScript-csomagméret; a kliensoldali renderelés keresőoptimalizálási és teljesítménybeli többletmunkát igényelhet [51].
Server-side rendering (SSR)	A szerver minden kérésnél előállítja az oldal HTML-válaszát [51].	Gyors első tartalommegjelenés, jobb keresőmotoros feldolgozhatóság.	Minden kérés szerveroldali számítást igényel; a kliensoldali interakció késhet a JavaScript betöltéséig.
Static Site Generation (SSG)	Az oldalak HTML-je build időben készül el, majd kész fájlként szolgálható ki [51].	Rendkívül gyors kiszolgálás, CDN-en (Content Delivery Network) jól gyorsítótárazható [38].	Gyakran változó vagy személyre szabott tartalom esetén minden módosítás új buildet igényelhet.

2.1. táblázat. Webes renderelési és architekturális megközelítések összehasonlítása

2.9. Webfejlesztési módszertanok

A modern webfejlesztésben a módszertani választás közvetlenül befolyásolja a tervezést, a fejlesztés ütemezését, a hibamegelőzést és a deployment módját [52, 6]. A Waterfall lineáris és erősen terv-vezérelt megközelítés, míg az Agile egy iteratív, visszajelzés-alapú szemlélet [53, 39]. Ennek két gyakori keretrendszere a Scrum és a Kanban [54, 55]. A DevOps ehhez képest nem pusztán projektmenedzsment-módszer, hanem egy olyan működési modell, amely a fejlesztést, a tesztelést és az üzemeltetést különböző automatizációkkal köti össze, különösen a CI/CD (Continuous Integration/Continuous Delivery vagy Continuous Deployment) használatával [56, 57].

2.9.1. Összehasonlítás

Az alábbi táblázat a legfontosabb modellek és módszertanok lényegét foglalja össze. Fontos különbség, hogy az Agile inkább szemlélet, a Scrum és a Kanban ennek konkrét megvalósítási formái, a DevOps pedig a kitelepítési és üzemeltetési folyamatokat egészíti ki automatizációval.

Módszer	Alapelv	Előnyök	Hátrányok
Waterfall	Egymást követő, lezárt fázisok: követelmények, tervezés, implementáció, verifikáció és karbantartás [52, 53].	Jó dokumentálhatóság, stabil ütemezés, előre tervezhető költségek.	Kevésbé rugalmas; újratervezés esetén költségessé és időigényessé válhat.
Agile	Iteratív, adaptív fejlesztési szemlélet, kisebb lépésekben történő szállítással [39, 53].	Gyors visszajelzés, könnyebb alkalmazkodás, erősebb ügyfél- és csapatkapcsolat.	Csapatfegyelmet igényel; kisebb lehet az előreláthatóság, és a dokumentáció háttérbe szorulhat.
Scrum	Időkeretes sprintek, definiált szerepkörök, események és produktumok [54].	Átlátható működés, rendszeres review, kiszámítható iterációk.	Több kötelező esemény és adminisztráció; szerepköri konfliktusok és meredekebb tanulási görbe jelentkezhet.
Kanban	Folyamatos áramlás, vizualizált munka és korlátozott folyamatban lévő feladatmennyiség [55].	Rugalmas prioritáskezelés, kevés felesleges adminisztráció, jól követhető kapacitás.	Kevésbé kényszerít tervezési ciklusokra; hosszú távon könnyebben elveszhet a ritmus.

2.2. táblázat. Gyakori webfejlesztési módszertanok összehasonlítása

2.10. Modern webfejlesztői csapat felépítése

Egy modern webfejlesztői csapat több, egymást kiegészítő szerepkörből áll. A csapat célja nem csupán a forráskód megírása, hanem egy teljes webalkalmazás megtervezése, megvalósítása, tesztelése, telepítése és hosszú távú karbantartása. Emiatt a fejlesztési folyamatban technikai, tervezési, minőségbiztosítási és üzemeltetési feladatok is megjelennek [52, 56].

A folyamat elején általában a product owner vagy projektmenedzser határozza meg az üzleti célokat, a prioritásokat és a fejlesztendő funkciókat [54]. Ő tartja kapcsolatot az ügyféllel vagy a meg-

rendelővel, és segít abban, hogy a csapat ne csak technikailag működő, hanem valódi felhasználói igényeket kielégítő és problémákat megoldó rendszert készítsen.

A felhasználói élmény és a felület megtervezéséért jellemzően a UX/UI designer felel. A UX tervezés a felhasználói útvonalakra, a használhatóságra és az alkalmazás logikus működésére koncentrál, míg a UI tervezés a vizuális megjelenést, az elrendezést, a színeket, a tipográfiát és az interakciós elemeket alakítja ki [58]. Egy jól megtervezett felület csökkenti a fejlesztési bizonytalanságot, mert a fejlesztők pontosabb képet kapnak arról, mit kell megvalósítaniuk.

A frontend fejlesztő a böngészőben futó kliensoldali részt készíti el. Ide tartozik a felhasználói felület HTML, CSS és JavaScript alapú megvalósítása, valamint modern keretrendszerek, például React, Angular vagy Vue használata [14, 17]. A frontend fejlesztő feladata, hogy a felület reszponzív, gyors, hozzáférhető és felhasználóbarát legyen, miközben megfelelően kommunikál a backend által biztosított API-kkal [4, 59].

A backend fejlesztő a szerveroldali logikáért felel. Feladata az üzleti logika megvalósítása, az adatbázis-kezelés, az autentikáció, az autorizáció, valamint az API-k kialakítása [2, 49]. A backend biztosítja, hogy a rendszer megbízhatóan kezelje az adatokat, megfelelő válaszokat adjon a kliensoldali kérésekre, és biztonságosan működjön több felhasználó vagy nagyobb terhelés mellett is (skálázhatóság).

Nagyobb csapatokban megjelenhet a full-stack fejlesztő szerepköre is, aki frontend és backend feladatokat egyaránt képes ellátni [60]. Ez különösen kisebb projektekben hasznos, ahol nincs külön ember minden technológiai rétegre. Ugyanakkor nagyobb rendszereknél a specializáció általában hatékonyabb, mert a kliensoldali és szerveroldali fejlesztés is külön mély szakértelmet igényel.

A minőségbiztosításért a QA engineer vagy tesztelő felel. Ő manuális és automatizált tesztekkel ellenőrzi, hogy az alkalmazás megfelel-e a követelményeknek, és nem tartalmaz-e kritikus hibákat. A tesztelés nem csupán a fejlesztés végén fontos, hanem a teljes folyamat során, mivel a korán felismert hibák javítása általában olcsóbb és egyszerűbb [61]. Különösen fontosak a regressziós hibák kiszűrésére szolgáló tesztek, mivel egy új fejlesztés vagy hibajavítás könnyen elronthat egy korábban már működő funkciót [62]. Az ismételhető, automatizált tesztek ezt a kockázatot csökkentik [63].

A DevOps engineer vagy üzemeltetési szakember a fejlesztés és az infrastruktúra közötti kapcsolatot biztosítja. Feladata lehet a CI/CD folyamatok kialakítása, a konténerizáció, a felhőalapú infrastruk-

túra kezelése, a monitoring, a naplózás és az automatikus telepítések beállítása [56, 57]. A DevOps szemlélet célja, hogy a fejlesztés, tesztelés és telepítés gyorsabb, megbízhatóbb és automatizáltabb legyen.

A modern webfejlesztői csapat működése erősen együttműködésre épül [39]. A szerepkörök nem teljesen elszigeteltek: a frontend fejlesztőnek értenie kell az API-k működését, a backend fejlesztőnek figyelembe kell vennie a felhasználói igényeket, a designernek ismernie kell a technikai korlátokat, a tesztelőnek pedig már a fejlesztés korai szakaszában részt kell vennie a követelmények pontosításában. Ezért a hatékony webfejlesztés nemcsak technológiai, hanem kommunikációs és szervezési feladat is.

2.11. Modern webalkalmazások fejlesztési folyamata

A modern webalkalmazások fejlesztése általában a szoftverfejlesztési életciklus, vagyis az SDLC (Software Development Life Cycle) lépéseire épül. Az SDLC célja, hogy a fejlesztés előre meghatározott, átlátható lépések mentén haladjon: a csapat először meghatározza a követelményeket, majd megtervezi, megvalósítja, teszteli, telepíti és hosszú távon karbantartja az alkalmazást [52]. Webalkalmazások esetében ez a folyamat különösen fontos, mivel a rendszer több rétegből áll: frontendből, backendből, adatbázisból, API-kból, infrastruktúrából és sok esetben külső szolgáltatásokból is [49, 2].

A fejlesztési folyamat első szakasza a követelmények összegyűjtése és pontosítása [64]. Ebben a fázisban a csapat meghatározza, hogy az alkalmazás milyen problémát old meg, kik lesznek a felhasználói, milyen funkciókra van szükség, és milyen üzleti vagy technikai korlátokat kell figyelembe venni. Ide tartozhatnak a funkcionális követelmények, például a felhasználói regisztráció, keresés vagy adminisztrációs felület, valamint a nem funkcionális követelmények is, például a teljesítmény, biztonság, skálázhatóság, hozzáférhetőség és reszponzív működés [65, 59].

A következő lépés a tervezés, amely során kialakul az alkalmazás architektúrája és felhasználói felülete [66, 58]. A UX/UI tervezés meghatározza a felhasználói útvonalakat, a képernyők felépítését és az interakciók logikáját, míg a technikai tervezés a frontend és backend feladatok szétválasztására, az API-k kialakítására, az adatmodellre és az infrastruktúrára koncentrál [49]. Egy jól átgondolt tervezési szakasz csökkenti a későbbi félreértéseket, és segít abban, hogy a fejlesztők egységes képet

kapjanak a megvalósítandó rendszerről.

Az implementáció során a fejlesztők elkészítik az alkalmazás működő részeit. A frontend fejlesztés a böngészőben futó felületre, a backend fejlesztés pedig a szerveroldali logikára, adatkezelésre, autentikációra, autorizációra és API-kra koncentrál [14, 2]. A modern webfejlesztésben gyakori, hogy a fejlesztés iteratív módon történik: a csapat kisebb funkciókat valósít meg, ezeket folyamatosan teszteli, majd visszajelzések alapján továbbfejleszti [39].

A tesztelés a fejlesztési folyamat egyik kulcsfontosságú része. Nemcsak a kész alkalmazás ellenőrzését jelenti, hanem a teljes fejlesztés során jelen van [61]. A manuális tesztek mellett egyre nagyobb szerepet kapnak az automatizált tesztek, például az egységtesztek, integrációs tesztek és end-to-end tesztek [67]. Ezek segítenek kiszűrni a regressziós hibákat is, amikor egy új fejlesztés vagy hibajavítás elront egy korábban már működő funkciót [62].

A telepítés és üzemeltetés szakaszában az alkalmazás kikerül a felhasználók számára elérhető környezetbe. Modern fejlesztési környezetekben ezt gyakran CI/CD folyamatok támogatják, amelyek automatizálják a buildelést, tesztelést és telepítést [57]. A DevOps szemlélet célja, hogy a fejlesztés és az üzemeltetés ne különálló, egymástól elszigetelt tevékenység legyen, hanem egységes, együttműködő folyamat [56].

Az SDLC utolsó, de folyamatosan visszatérő része a karbantartás és továbbfejlesztés [68]. Egy webalkalmazás a kiadás után sem tekinthető befejezettnek: hibajavításokra, biztonsági frissítésekre, teljesítményoptimalizálásra és új funkciók bevezetésére is szükség lehet [69]. Emiatt a modern webalkalmazás-fejlesztés inkább ciklikus folyamatként értelmezhető, ahol a tervezés, fejlesztés, tesztelés és üzemeltetés folyamatosan ismétlődik.

2.12. A választott technológiai stack áttekintése

A megvalósított alkalmazás TypeScript alapú Next.js keretrendszerre épül, amely a React komponensalapú felépítését szerveroldali rendereléssel, fájlrendszer-alapú útvonalkezeléssel és API útvonalakkal egészíti ki [70, 71]. Ennek köszönhetően a frontend és a backend jellegű logika egy közös projektstruktúrában jelenik meg: a felhasználói felület React komponensekből épül fel, míg az adatok lekérése és módosítása Next.js API route-okon keresztül történik.

Az alkalmazás adatkezeléséhez és autentikációjához Supabase szolgáltatást használ. A Supabase kliens külön szerveroldali és böngészőoldali inicializálással jelenik meg, ami illeszkedik a Next.js szerveroldali működéséhez és a felhasználói munkamenetek kezeléséhez [72]. A rendszer saját adatbázisa elsősorban a felhasználói állapotokat tárolja, például a watchlist és watched listák elemeit, míg a filmek részletes adatai külső forrásból, a TMDB API-ból érkeznek [73].

A kliensoldali adatlekéréseket és a cache-t a TanStack Query kezeli, amely az aszinkron szerverállapot frissítését és szinkronizálását támogatja React alkalmazásokban [74]. A felhasználói felület kialakításához Tailwind CSS, shadcn/Radix UI komponensek és lucide-react ikonok kerültek felhasználásra. A Tailwind utility-first megközelítést biztosít a stílusok gyors és egységes kialakításához [75]. A Radix UI és a shadcn komponensek újrafelhasználható, hozzáférhetőségi szempontokat is figyelembe vevő felületi elemek készítését támogatják [76, 77].

3. fejezet

Projektcélok és termékkonceptió

A következő fejezet célja annak bemutatása, hogy a fejlesztés nem közvetlenül az implementációval indult, hanem egy előzetes terméktervezési folyamattal, amelynek központi eleme a Product Requirements Document (PRD).

A PRD alapján meghatározhatóvá vált a projekt kiinduló problémája, célközönsége, fő tervezett funkciói, valamint az első megvalósítható verzió, vagyis az MVP-hez (Minimum Viable Product) szükséges követelmények.

3.1. A projekt kiinduló problémája

Manapság az emberek filmnézési szokásai több platformra terjednek ki: streaming szolgáltatókra, mozikra, fizikai adathordozókra és televíziós tartalmakra. Ezek a megoldások rendkívül kézenfekvővé teszik a filmek fogyasztását, ugyanakkor egy új problémát is létrehoznak. Egy adott szolgáltatás, például a Netflix, képes nyilvántartani, hogy a felhasználó az adott platformon milyen filmeket nézett meg, vagy mit szeretne később megnézni, ez azonban nem terjed ki más szolgáltatásokra.

Emiatt a felhasználó filmnézési előzményei nehezen követhetővé válnak. Előfordulhat, hogy valaki nem emlékszik pontosan, látott-e már egy filmet, milyen értékelést adott volna rá, vagy mely filmeket szerette volna később megnézni. A probléma tehát nem kizárólag a filmek megtalálásáról szól, hanem egy személyes, platformfüggetlen filmnapló kialakításáról is.

A filmnézésnek ugyanakkor közösségi oldala is van. Gyakran kíváncsiak vagyunk arra, hogy

ismerőseink milyen filmeket néztek meg az utóbbi időben, mit gondoltak róluk, vagy milyen filmeket ajánlanának. Ezek az ajánlások sokszor beszélgetésekben, csoportos üzenetekben vagy közösségi médiás bejegyzésekben jelennek meg, így könnyen elvesznek, és később nehezen kereshetők vissza. A legtöbb streaming platform elsősorban a saját katalógusára és az egyéni fogyasztásra koncentrál, ezért nem ad teljes választ arra, hogyan lehetne a személyes filmnaplót és a baráti ajánlásokat egységes rendszerben kezelni.

A WatchDB projekt kiinduló problémája ezért kettős. Egyrészt szükség van egy egyszerű, platformfüggetlen felületre, ahol a felhasználó nyilvántarthatja a megnézett filmeket és a később megtekintendő alkotásokat. Másrészt megjelenik az igény egy olyan közösségi rétegre is, amelyen keresztül a felhasználók követhetik egymás filmnézési aktivitását és ajánlásokat fedezhetnek fel. A dolgozatban bemutatott projekt ezt a problémát használja kiindulópontként, majd ebből vezeti le a termék koncepcióját, a követelményeket és az MVP-hez szükséges funkciókat.

3.2. A PRD szerepe a fejlesztési folyamatban

A Product Requirements Document (PRD) a projekt előkészítésének egyik legfontosabb dokumentuma. Szerepe, hogy a termékötletet strukturált formába rendezze, és összekapcsolja a felhasználói igényeket, az üzleti célokat, a funkcionális követelményeket és a fejlesztési korlátokat. A PRD így nem pusztán leíró dokumentum, hanem a fejlesztési folyamat egyik kiindulópontja: segít meghatározni, hogy milyen problémát kell megoldani, kinek készül a termék, milyen funkciók szükségesek az első verzióhoz, és mely elemek kerülhetnek későbbi fejlesztési szakaszba.

A PRD elkészítése ideális esetben nem egyetlen személy feladata, hanem több szerepkör közös munkájának eredménye. A termékoldali szereplők a felhasználói igényeket és az üzleti célokat képviselik, a fejlesztők a technikai megvalósíthatóságot és a becsült ráfordítást vizsgálják, a tervezői és minőségbiztosítási szempontok pedig a használhatóságot, tesztelhetőséget és a követelmények egyértelműségét erősítik. Ez azért fontos, mert egy jól elkészített PRD nemcsak azt rögzíti, hogy mit kell megvalósítani, hanem azt is, hogy a csapat miért éppen ezeket a funkciókat választotta.

A WatchDB esetében a PRD különösen fontos szerepet tölt be, mert a kezdeti termékvízió szélesebb, mint az elsőként megvalósítható alkalmazás. A dokumentum alapján a projekt nemcsak szemé-

lyes filmnaplóként értelmezhető, hanem hosszabb távon közösségi filmkövető platformként is. Ez a vízió több irányt tartalmaz: a felhasználó saját filmjeinek naplózását, a watchlist kezelését, értékelések rögzítését, valamint későbbi verziókban a barátok aktivitásának követését és a közösségi ajánlások megjelenítését.

A PRD egyik legfontosabb feladata ebben a helyzetben a kezdeti termék terjedelmének a kezelése. Egy teljes termékvízió gyakran több olyan funkciót tartalmaz, amelyek egyszerre történő megvalósítása túl nagy fejlesztési terhet jelentene. Ezért a dokumentum a funkciókat prioritás szerint csoportosítja: az MVP-hez szükséges funkciókra, a későbbi verziókban megvalósítható elemekre, valamint azokra a részekre, amelyek tudatosan kívül maradnak a projekt hatókörén. Ez a felosztás segít elkerülni, hogy az első verzió túl összetetté váljon, miközben megőrzi a termék hosszabb távú fejlesztési irányát.

A WatchDB PRD-je alapján az MVP központi célja, hogy a felhasználó gyorsan és egyszerűen tudjon filmeket keresni, megnézett filmeket naplózni, értékelést adni, valamint külön listában vezetni azokat a filmeket, amelyeket később szeretne megtekinteni. Ezek a funkciók alkotják azt a minimális terméket, amely már önmagában is értéket ad a felhasználónak, ez az MVP (Minimum Viable Product) lényege. A közösségi funkciók, például a publikus profilok, a követési rendszer vagy az aktivitási feed, a termékvízió fontos részei, de az első megvalósítható verzió szempontjából későbbi fejlesztési szakaszba sorolhatóak.

A PRD további előnye, hogy a fejlesztési döntéseket később visszakereshetővé és indokolhatóvá teszi. A célközönség, a felhasználói folyamatok, a nem funkcionális követelmények, a technológiai függőségek és a korlátok mind olyan tényezők, amelyek befolyásolják az alkalmazás tervezését. Például a külső filmdatok használata indokolja a TMDB API integrációját, míg az email-alapú bejelentkezés az MVP egyszerűsítését szolgálja.

Összességében a PRD a WatchDB fejlesztésében hidat képez a termékötlet és a technikai megvalósítás között. Segítségével a projekt nem elszigetelt funkciók gyűjteményeként jelenik meg, hanem olyan tudatosan szűkített fejlesztési feladatként, amely egy nagyobb termékvízió első, megvalósítható lépését képviseli és egy olyan irányt mutató dokumentum, amihez a csapat könnyen tud igazodni és utalni rá a fejlesztési folyamat során.

3.3. Termékvízió

A WatchDB termékvíziója egy olyan webalkalmazás létrehozása, amely egyszerre működik személyes filmnaplóként és később közösségi filmfelfedező platformként. A projekt alap gondolata, hogy a felhasználó egyetlen, platformfüggetlen helyen tudja rögzíteni, milyen filmeket nézett meg, miket szeretne később megnézni, és hogyan értékelte a már látott filmeket. Ez a megközelítés elsősorban a filmnézési előzmények rendszerezésére és a személyes filmnapló kialakítására épül.

A termék egyik célja a személyes használat. Ebben a felhasználó filmeket kereshet, részletes információkat tekinthet meg róluk, hozzáadhatja őket a watchlisthez, illetve megnézettként jelölheti és értékelheti őket. Ez a funkciókör adja a termék alapértékét, mert akkor is hasznos, ha a felhasználó egyedül használja az alkalmazást. A cél az, hogy a filmek naplózása gyors és egyszerű legyen, ne igényeljen hosszú szöveges értékeléseket, és ne legyen bonyolultabb annál, mint amit egy hétköznapi filmnéző szívesen használna.

A termék második, hosszabb távú célja a közösségi használat. A PRD alapján a WatchDB nemcsak azt tenné lehetővé, hogy a felhasználó saját filmjeit nyilvántartsa, hanem azt is, hogy lássa, barátai vagy ismerősei milyen filmeket néztek meg, hogyan értékelték azokat, illetve milyen filmeket tettek a saját listáikra. Ez a közösségi dimenzió a személyes filmnaplót ajánlási és felfedezési felületté bővítené: a felhasználó nem algoritmikus ajánlórendszeren keresztül, hanem ismerőseinek aktivitása alapján találhatna új filmeket.

Fontos különbséget tenni a teljes termékvízió és az elsőként megvalósítandó MVP között. A teljes vízió része lehet a publikus profil, a követési rendszer, az aktivitási feed, a reakciók, a statisztikai nézetek vagy akár egy fejlettebb ajánlórendszer is. Ezek azonban nem feltétlenül szükségesek ahhoz, hogy az alkalmazás első verziója értéket adjon. Az MVP ezért a termék személyes filmnapló funkcióira koncentrál: a keresésre, a filmrészletek megjelenítésére, a watchlist kezelésére, a megnézett filmek naplózására és az értékelésre.

3.4. Célközönség és perszónák

A célközönség meghatározása a termékkonceptió egyik fontos része, mert a fejlesztési döntések nem választhatók el attól, hogy kiknek készül az alkalmazás. A PRD alapján a termék elsődleges célcsoportját azok az alkalmi filmnézők alkotják, akik havonta néhány filmet néznek, és egyszerű, gyors megoldást keresnek a filmjeik nyilvántartására. Másodlagos célcsoportként megjelennek azok a filmrajongók is, akik tudatosabban követik filmnézési szokásaikat, és hosszabb távon részletesebb statisztikákat vagy közösségi funkciókat is használnának.

Az első célszemély az alkalmi filmkövető típusa. Ő rendszeresen néz filmeket streaming szolgáltatásokon vagy moziban, de nem feltétlenül tekinti magát filmrajongónak. Számára a legfontosabb érték az egyszerűség, gyorsan szeretné rögzíteni, hogy mit látott már, mit szeretne később megnézni, és esetleg milyen benyomása volt egy adott filmről. Ez a felhasználói típus indokolja, hogy az MVP ne legyen túlterhelt bonyolult funkciókkal, hosszú értékelési lehetőségekkel vagy összetett közösségi funkciókkal. A keresésnek, a megnézendő filmek listájának kezelésének és a megnézett filmek naplózásának ezért kevés lépésből kell állnia.

A második célszemély a közösségi filmnéző, aki nemcsak saját magának vezetne listát, hanem kíváncsi arra is, hogy ismerősei milyen filmeket néznek és hogyan értékelik azokat. Ennél a felhasználói típusnál a filmnézés egy bizonyos szociális aspektussá is válik. A PRD-ben szereplő későbbi közösségi funkciók, például a publikus profil, a követési rendszer és az aktivitási feed ehhez a célcsoporthoz kapcsolódik. Ezek a funkciók nem feltétlenül részei az első MVP-nek, de fontosak a termék számára hosszútávon.

A harmadik célszemély a tudatos filmrajongó vagy filmkedvelő, aki nagyobb mennyiségben néz filmeket, szívesen értékel és rendszerez, de nem feltétlenül szeretne hosszú kritikákat írni. Számára a személyes profil, a megtekintett filmek listája, az értékelések és a későbbi statisztikai kimutatások bizonyulhatnak értékesnek. Ez a perszóna bemutatja, hogy a termékvízióban a filmnapló mellett megjelenhetnek mélyebb funkciók is, például műfaji statisztikák, éves összegzések, egyéni listák vagy ajánlási lehetőségek.

A célszemélyek közös jellemzője, hogy mindhárom esetben fontos a gyors használhatóság és az átlátható felület. A különbség inkább abban jelenik meg, hogy az egyes felhasználói típusok milyen

mélységben használnák az alkalmazást. Az alkalmi felhasználó számára az MVP alapfunkciói is elegendő értéket adhatnak, míg a közösségi filmnéző és a filmrajongó inkább a későbbi verziókban megjelenő bővítéseket tartaná hasznosnak. Ez a felosztás segít abban, hogy az első fejlesztési szakasz a legszélesebb célcsoport számára hasznos funkciókra koncentráljon, miközben a termék jövőbeli bővíthetőségére is lehetőség adódik.

Ennek alapján a WatchDB célközönsége nem egyetlen, szűken meghatározott, hanem több, eltérő igényű és filmnézési szokásokkal rendelkező csoportból áll. A PRD-ben bemutatott célszemélyek szerepe ezért az, hogy a funkciók fontossági sorrendje először a legnagyobb célközönség alapvető igényeihez igazodjon, és csak ezt követően kerüljenek előtérbe a szűkebb felhasználói csoportokat kiszolgáló bővítések.

3.5. Piaci kontextus

A WatchDB termékkonceptiójának értelmezéséhez érdemes röviden áttekinteni azokat a meglévő megoldásokat, amelyek részben hasonló problémára adnak választ. A piaci kontextus célja ebben az esetben nem egy teljes versenytárselemzés, hanem annak bemutatása, hogy a filmek követése, listázása és megosztása már létező felhasználói igény, ugyanakkor a különböző szolgáltatások eltérő megoldásokkal közelítik meg ezt a problémát.

Az IMDb egyik fontos releváns funkciója a watchlist. A felhasználók címeket adhatnak a saját watchlistjükhöz, rendezhetik azt, illetve értékelések és listák alapján is fedezhetnek fel új tartalmakat [78]. Ez a megoldás jól mutatja, hogy a filmek későbbi megtekintésre való elmentése alapvető igény. A WatchDB szempontjából ugyanakkor az IMDb inkább információközpontú példaként jelenik meg, míg a projekt célja egy letisztultabb, személyes filmnapló és később közösségi élmény köré szerveződő alkalmazás.

A Trakt erősebb követési és integrációs szemléletet képvisel. A szolgáltatás filmek és sorozatok követésére, listák kezelésére, ajánlásokra, valamint különböző alkalmazásokkal és médiaközpontokkal való összekapcsolásra épít [79]. Ez a megközelítés funkcionálisan erős, ugyanakkor összetettebb termékélményt eredményezhet. A WatchDB PRD-je ezért inkább egy egyszerűbb, filmközpontú induló verziót határoz meg, ahol az elsődleges cél nem a sok integráció, hanem a gyors filmkeresés,

naplózás és listaépítés.

A piacon jelen vannak kimondottan közösségi filmnapló platformok is, például a Letterboxd. A Letterboxd a filmek nyilvántartását, értékelését, naplózását, listázását és a barátok követését is támogatja, vagyis erősen kapcsolódik ahhoz a közösségi irányhoz, amely a WatchDB hosszabb távú víziójában is megjelenik [80]. Ez azt mutatja, hogy a filmnapló és a közösségi filmfelfedezés kombinációja valós felhasználói igényre épül. A WatchDB dolgozatbeli szerepe azonban nem az, hogy minden létező hasonló szolgáltatással teljes funkcionalitásban versenyezzen, hanem az, hogy egy ilyen termék tervezési és fejlesztési folyamatát egy szűkített MVP-n keresztül mutassa be.

A PRD-ben szereplő piaci összehasonlítás alapján a WatchDB lehetősége abban áll, hogy a fenti irányokból egy tudatosan szűkített változatot határoz meg. Az IMDb adatgazdagságából, a Trakt követési szemléletéből, a streaming szolgáltatások kényelmes listakezeléséből és a Letterboxd közösségi megközelítéséből is levonhatók tanulságok. Az MVP szempontjából azonban ezek közül azok a funkciók kapnak elsőbbséget, amelyek a legszélesebb célközönség számára azonnali értéket adnak: filmkeresés, watchlist, megnézett filmek naplózása és egyszerű értékelés. A közösségi és haladó funkciók így nem elvetett elemek, hanem a termék későbbi fejlődési irányai.

3.6. Termékcélok

A termékcélok meghatározása a PRD egyik kulcsfontosságú része, mert ezek jelölik ki, hogy a fejlesztés során milyen értéket kell létrehozni a felhasználók számára. A célok nem azonosak a konkrét funkciókkal: inkább azt fogalmazzák meg, hogy az alkalmazásnak milyen problémára kell megoldást adnia, milyen felhasználói élményt kell támogatnia, és milyen szempontok alapján lehet eldönteni, hogy az első verzió sikeresnek tekinthető-e.

A WatchDB elsődleges termékcélja, hogy a felhasználó gyorsan és egyszerűen tudjon filmeket keresni, majd azokat saját listáihoz hozzáadni. Ez a cél közvetlenül kapcsolódik a projekt kiinduló problémájához: a filmnézési élmények több platformon és helyzetben történnek, ezért a felhasználónak szüksége van egy egységes, platformfüggetlen nyilvántartásra. Az MVP-ben ennek a célnak a filmkeresés, a filmrészletek megjelenítése, a watchlist és a megnézett filmek naplózása felel meg.

A második fontos cél a személyes filmnapló felépítése. A WatchDB nem csupán egy megnézendő

filmeket tartalmazó lista, hanem olyan személyes filmnapló, amelyben a felhasználó visszakeresheti, milyen filmeket látott, hogyan értékelte őket, és milyen filmeket szeretne később megnézni. Ez a cél azért fontos, mert az alkalmazásnak akkor is értéket kell adnia, ha a felhasználó egyelőre nem használ közösségi funkciókat. Az MVP tehát önmagában is használható termékként jelenik meg, nem csak egy későbbi, nagyobb rendszer előzetes változataként.

A harmadik termékcél a közösségi filmfelfedezés előkészítése. A teljes termékvízió alapján a WatchDB hosszabb távon lehetőséget adna arra, hogy a felhasználók kövessék egymás aktivitását, megtekintsék barátaik értékeléseit, és ezek alapján fedezzenek fel új filmeket. Ez a cél az MVP-ben még nem feltétlenül jelenik meg teljes funkcionalitásként, de fontos tervezési szempontként hat a rendszerre. A felhasználói profil, a naplózott filmek strukturált tárolása és a később bővíthető adatmodell mind olyan elemek, amelyek támogatják ezt az irányt.

A negyedik cél a gyors, rezponzív és könnyen használható felhasználói élmény biztosítása. Mivel a célközönség jelentős része alkalmi filmnéző, az alkalmazásnak nem szabad túl sok döntést vagy bonyolult lépést kérnie a felhasználótól. A film keresése, listához adása vagy értékelése akkor működik jól, ha néhány kattintással elvégezhető. Ez a cél nemcsak felületi kérdés, hanem technikai döntéseket is befolyásol: fontos a gyors oldalbetöltés, a hatékony keresés, a mobilon is használható felület és az egyértelmű navigáció.

A PRD-ben szereplő sikerességi mutatók ezekhez a célokhoz kapcsolódnak. Ilyen mutató lehet például a regisztrált felhasználók száma, az aktív felhasználók aránya, az egy felhasználóra jutó havi naplózott filmek száma, a kereséstől a naplózásig eltelt idő, illetve későbbi verzióban az is, hogy hány felhasználó kapcsolódik legalább egy ismerőshöz. Ezek a mérőszámok azért hasznosak, mert a termékcélokat ellenőrizhetővé teszik: nemcsak azt mutatják meg, hogy elkészültek-e a funkciók, hanem azt is, hogy a felhasználók valóban használják-e őket. Emellett a termékcélok a későbbi iteratív fejlesztésben is támpontot adhatnak: segíthetnek eldönteni, hogy érdemes-e új funkcióval bővíteni az alkalmazást, vagy éppen azt, hogy egy kevésbé használt funkció fenntartása továbbra is indokolt-e.

Összességében a WatchDB termékcéljai a személyes értékteremtésre és a későbbi közösségi bővíthetőségre épülnek. Az első verzióban a hangsúly a gyors filmkeresésen, a watchlist kezelésén, a megnézett filmek naplózásán és az egyszerű értékelésen van. Ezek a célok elég szűkek ahhoz, hogy egy MVP keretében megvalósíthatók legyenek, ugyanakkor elég stabil állapotot adnak ahhoz, hogy a

termék később közösségi filmfelfedező platformmá fejlődhessen.

3.7. Sikerességi metrikák

A termékcélok mellett a PRD sikerességi metrikákat is meghatároz. Ezek szerepe, hogy a célokat mérhetőbbé tegyék, és a fejlesztés későbbi szakaszaiban visszajelzést adjanak arról, hogy az alkalmazás valóban betölti-e a tervezett szerepét. Egy funkció elkészülése önmagában még nem jelenti azt, hogy a termék sikeres: azt is vizsgálni kell, hogy a felhasználók megtalálják-e az alkalmazást, rendszeresen visszatérnek-e, és ténylegesen használják-e azokat a funkciókat, amelyekre az MVP épül.

A WatchDB esetében a sikerességi mutatók három fő terület köré rendezhetők. Az első a felhasználói növekedés, amely azt mutatja meg, hogy az alkalmazás képes-e elérni egy kezdeti felhasználói bázist. A második az aktivitás és megtartás, amely azt vizsgálja, hogy a regisztrált felhasználók visszatérnek-e, illetve rendszeresen naplóznak-e filmeket. A harmadik a használati élmény és teljesítmény, amely azt méri, hogy a legfontosabb felhasználói folyamatok, például a filmkeresés és a naplózás, elég gyorsan és gördülékenyen működnek-e.

Metrika	Cél 3 hónap után	Cél 6 hónap után
Regisztrált felhasználók száma	500 felhasználó	2000 felhasználó
Aktív felhasználónként naplózott filmek száma havonta	Legalább 5 film	Legalább 8 film
Heti aktív felhasználók aránya	A regisztrált felhasználók 30%-a	A regisztrált felhasználók 40%-a
Legalább egy ismerőssel rendelkező felhasználók aránya	Nem elsődleges MVP-metrika	A felhasználók 60%-a
Kereséstől naplózásig eltelt átlagos idő	< 15 másodperc	< 10 másodperc
Oldalbetöltési idő	< 2 másodperc	< 2 másodperc

3.1. táblázat. A WatchDB PRD-ben meghatározott sikerességi metrikái

A regisztrált felhasználók száma elsősorban azt jelzi, hogy a termék képes-e érdeklődést kiváltani a célközönségből. Ez a metrika azonban önmagában nem elegendő, mert egy alkalmazás használhatóságát és valódi értékét nem pusztán a regisztrációk száma mutatja meg. Emiatt fontosabb visszajelzést adhat az aktív felhasználók aránya és a naplózott filmek száma. Ha a felhasználók havonta több filmet is rögzítenek, az arra utal, hogy az alkalmazás valóban beépülhet a filmnézési szokásaikba.

A kereséstől a naplózásig eltelt idő különösen fontos metrika az MVP szempontjából. A WatchDB egyik alapcélja, hogy a filmek rögzítése gyors és kevés lépésből álló folyamat legyen. Ha ez az idő túl hosszú, az arra utalhat, hogy a keresési felület, a filmrészletek oldala vagy a naplózási művelet túl bonyolult. Ez közvetlenül visszahat a felhasználói élményre, és indokolhatja a felület vagy a felhasználói folyamat egyszerűsítését.

A teljesítményhez kapcsolódó metrikák, például az oldalbetöltési idő, azért fontosak, mert a célközönség jelentős része hétköznapi, gyors használatra alkalmas alkalmazást vár. Egy film keresése vagy listához adása gyakran rövid, spontán művelet, ezért a lassú betöltés vagy akadozó felület könnyen csökkentheti a használati hajlandóságot. A két másodperc alatti oldalbetöltési cél ezért nem csupán

technikai elvárás, hanem a termék használhatóságához kapcsolódó követelmény is.

A közösségi metrikák, például az ismerősökhöz kapcsolódó felhasználók aránya, a PRD-ben inkább a későbbi termékverzióhoz kötődnek. Az MVP elsődleges célja még a személyes filmnapló működőképességének bizonyítása, ezért a közösségi kapcsolatok aránya az első három hónapban nem tekinthető alapvető sikerességi feltételnek. Hat hónapos távon viszont már fontos visszajelzést adhat arról, hogy a WatchDB képes-e túllépni az egyéni használaton, és valóban közösségi filmfelfedező platform irányába fejlődni.

Összességében a sikerességi metrikák nemcsak utólagos értékelésre szolgálnak, hanem a fejlesztési döntéseket is támogatják. Segítségükkel mérhetővé válik, hogy az MVP mely részei működnek jól, mely folyamatokon kell egyszerűsíteni, és mely későbbi funkciók bevezetése indokolt. Ez különösen fontos egy olyan termék esetében, amelynek hosszabb távú víziója szélesebb, mint az első megvalósított verzió.

3.8. MVP és verziókra bontás

A PRD egyik legfontosabb döntése az MVP, vagyis a Minimum Viable Product meghatározása. Ennek szerepe, hogy a teljes termékvízióból kiválassza azt a legkisebb, de már önmagában is értéket adó funkciókört, amely alapján az alkalmazás első verziója elkészíthető és kipróbálható. A WatchDB esetében ez különösen fontos, mert a hosszabb távú termékvízió személyes filmnaplót, közösségi aktivitást, ajánlásokat, statisztikákat és többféle listakezelési lehetőséget is tartalmaz. Ha ezek egyszerre kerülnének be az első fejlesztési szakaszba, az jelentősen növelné a projekt komplexitását.

Az MVP célja ezért nem az, hogy a WatchDB teljes vízióját azonnal megvalósítsa, hanem az, hogy bizonyítsa az alapötlet működőképességét. Az első verzióban a felhasználónak képesnek kell lennie regisztrálni, filmeket keresni, watchlistet vezetni, megnézett filmeket naplózni, értékelést adni, valamint saját profilján áttekinteni a rögzített filmeket. Ezek a funkciók közvetlenül kapcsolódnak a projekt kiinduló problémájához: a felhasználó egyetlen helyen szeretné követni, mit látott már, mit szeretne később megnézni, és hogyan értékelte a filmeket.

Verzió	Fő funkciók	Termékoldali szerep
MVP	Regisztráció és bejelentkezés, filmkeresés, watchlist, megnézett filmek naplózása, értékelés, saját profil	Az alapérték bizonyítása: a felhasználó önállóan is tud személyes filmnaplót vezetni.
v1.1	Publikus profilok, követési rendszer, aktivitási feed, reakciók, filmnapló idővonal	A közösségi réteg megalapozása, amely a személyes filmnaplót filmfelfedező felületté bővíti.
v1.2	Többszöri megtekintés kezelése, kommentek, egyéni listák, statisztikai nézetek, ajánlások, keresési szűrők, OAuth bejelentkezés	Mélyebb felhasználói élmény és nagyobb elköteleződés támogatása a már működő alrendszerre építve.
Hatókörön kívül	Sorozatkövetés, hosszú szöveges kritikák, streaming-elérhetőség, belső üzenetküldés, natív mobilalkalmazás, moderációs eszközök	A projekt fókuszának megőrzése, hogy az első verzió ne térjen el a személyes filmnapló és későbbi közösségi filmfelfedezés fő irányától.

3.2. táblázat. A WatchDB funkcióinak verziókra bontása

Az MVP-be kerülő funkciók kiválasztásánál az elsődleges szempont az volt, hogy az alkalmazás közösségi funkciók nélkül is használható legyen. Ez azért fontos, mert egy közösségi termék akkor működik jól, ha már van elegendő felhasználó és tartalom, amelyre a közösségi réteg épülhet. A WatchDB első verziója ezért először a személyes használati értéket teremti meg: a felhasználó saját filmjeit tudja rögzíteni, listázni és értékelni. Ez stabil alapot ad ahhoz, hogy később a barátok aktivitása, a publikus profilok és az ajánlási lehetőségek is értelmezhetővé váljanak.

A v1.1 verzió a PRD alapján a közösségi alapréteg kialakítására fókuszál. Ebben a szakaszban jelenhetnek meg a publikus profilok, a követési rendszer, az aktivitási feed és a reakciók. Ezek a funkciók már nem csupán a személyes nyilvántartást támogatják, hanem azt is lehetővé teszik, hogy a felhasználó mások filmnézési tevékenységéből inspirációt merítsen. Ez a lépés kapcsolja össze a termék személyes filmnapló jellegét a hosszabb távú közösségi filmfelfedező vízióval.

A v1.2 verzió a mélyebb használatot és az elköteleződést támogató funkciókat tartalmazza. Ide sorolhatók a többszöri megtekintések kezelése, a kommentek, az egyéni listák, a statisztikai nézetek, az ajánlások, a fejlettebb keresési szűrők és az OAuth alapú bejelentkezés. Ezek a funkciók hasznosak lehetnek, de nem szükségesek ahhoz, hogy az első verzió teljesítse az alapfeladatát. Emiatt későbbi fejlesztési szakaszba kerülnek, amikor az MVP működéséből és a felhasználói visszajelzésekből már pontosabban látható, mely bővítések indokoltak.

A hatókörön kívülre sorolt elemek legalább annyira fontosak, mint a megvalósítandó funkciók. A sorozatkövetés, a hosszú kritikák írása, a streaming-elérhetőség megjelenítése, a belső üzenetküldés vagy a natív mobilalkalmazás mind olyan irányok, amelyek önmagukban értékesek lehetnének, de eltérítenék a projektet az elsődleges céltól. Ezek kizárása segít abban, hogy a fejlesztés ne váljon túl szélessé, és az első verzió valóban a filmkeresésre, a watchlistre, a naplózásra és az értékelésre koncentráljon.

Ez a verziókra bontás a dolgozat szempontjából is lényeges, mert jól mutatja, hogy a projekt nem egyszerűen funkciók felsorolásaként épül fel, hanem tudatos terméktervezési döntések eredményeként. Az MVP kijelöli a megvalósítás első határát, a későbbi verziók pedig megőrzik a termék fejlődési irányát. Így a WatchDB nem lezárt végtermékként, hanem egy nagyobb termékvízió első, megvalósítható és értékelhető lépéseként jelenik meg.

3.9. A user flow-k szerepe

A user flow-k, vagyis felhasználói folyamatok szerepe az, hogy a PRD-ben meghatározott funkciókat konkrét használati helyzetekhez kapcsolják. Egy funkció önmagában csak azt írja le, hogy mire képes az alkalmazás, a user flow viszont azt mutatja meg, hogy a felhasználó milyen lépésekben jut el egy cél eléréséig. Ez különösen fontos az MVP tervezésekor, mert segít eldönteni, hogy mely képernyők, útvonalak és műveletek szükségesek ahhoz, hogy az első verzió valóban használható legyen.

A WatchDB esetében a user flow-k a termék legfontosabb használati eseteit modellezik. Az első folyamat egy új felhasználó első filmnaplózását mutatja be: regisztráció, keresés, film kiválasztása, megnézettként jelölés, értékelés, majd a profiloldalon való megjelenés. Ez a folyamat azért kiemelt

jelentőségű, mert végigvezeti a felhasználót azon az útvonalon, amely az alkalmazás alapértékét adja. Ha ez a folyamat túl hosszú, bizonytalan vagy nehezen követhető, akkor az MVP egyik legfontosabb célja sérül.

A második folyamat a watchlist használatára épül. Ebben a felhasználó bejelentkezés után filmet keres, hozzáadja azt a megnézendő filmek listájához, majd a profiloldalon ellenőrzi, hogy a film megjelent-e a listában. Ez a folyamat egyszerűnek tűnhet, de terméktervezési szempontból fontos, mert több alapvető funkciót összekapcsol: az autentikációt, a keresést, a filmadatok kezelését, a lista-állapot módosítását és a profilnézetet.

User flow	Fő cél	Kapcsolódó verzió
Első film naplózása	A felhasználó regisztráció után filmet keres, megnézettként jelöli és értékeli azt.	MVP
Film hozzáadása watchlisthez	A felhasználó későbbi megtekintésre elment egy filmet, majd ellenőrzi azt a profilján.	MVP
Film felfedezése ismerős aktivitásából	A felhasználó a feedben látott ismerős aktivitás alapján ment el egy filmet.	v1.1
Ismerős követése	A felhasználó megkeres egy másik profilt, követi azt, majd annak aktivitása megjelenik a feedben.	v1.1
Egyéni lista megosztása	A felhasználó saját listát készít, publikussá teszi, majd megosztja másokkal.	v1.2

3.3. táblázat. A WatchDB PRD-ben szereplő fő felhasználói folyamatok

A PRD-ben szereplő user flow-k jól elkülönítik az MVP és a későbbi verziók céljait. Az első két folyamat közvetlenül az MVP-hez kapcsolódik, mert a személyes filmnapló alapl működését írja le. Ezek alapján meghatározható, hogy az első verzióhoz szükség van regisztrációs és bejelentkezési felületre, keresőoldalra, filmrészletek megjelenítésére, watchlist műveletekre, naplózási és értékelési

lehetőségre, valamint egy saját profiloldalra. Így a user flow-k nemcsak a felhasználói élményt írják le, hanem közvetlenül segítik az oldaltérkép és a fejlesztési feladatok kialakítását is.

A harmadik és negyedik folyamat már a közösségi irányhoz kötődik. Az aktivitási feedből történő filmfelfedezés és az ismerős követése azt mutatja meg, hogyan válhat a WatchDB személyes filmnaplóból közösségi filmfelfedező platformmá. Ezek a folyamatok fontosak a termékvízió szempontjából, de az MVP hatókörén kívülre kerülnek, mert működésükhöz már publikus profilokra, követési rendszerre, aktivitási adatokra és feed logikára is szükség van.

Az ötödik folyamat, az egyéni lista megosztása, még későbbi bővítési irányt képvisel. Ez már nem az alapvető filmnaplózást, hanem a felhasználói tartalom szervezését és megosztását támogatja. A PRD-ben ezért a v1.2 verzióhoz kapcsolódik, ahol az alkalmazás már nemcsak követésre és naplózásra, hanem mélyebb rendszerezésre és közösségi megosztásra is alkalmas lehet.

A user flow-k fejlesztési szempontból azért is hasznosak, mert ellenőrizhetővé teszik a funkciók működését. Egy-egy folyamat alapján könnyebben megfogalmazhatók az elfogadási feltételek: például sikeres-e a regisztráció után a keresés, megjelenik-e a naplózott film a profiloldalon, vagy a watchlisthez adott film valóban visszakereshető-e. Ez a minőségbiztosítás szempontjából is fontos, mert a tesztelés nem elszigetelt komponensekre, hanem valós felhasználói útvonalakra épülhet.

Összességében a user flow-k a WatchDB tervezésében hidat képeznek a termékkonceptió és az implementáció között. Segítenek abban, hogy a fejlesztés ne pusztán funkciók megvalósítására koncentráljon, hanem arra, hogy a felhasználó ténylegesen el tudja érni a számára fontos célokat. Emiatt a user flow-k nem kiegészítő elemei a PRD-nek, hanem a döntések egyik gyakorlati alapját adják.

3.10. Korlátok és függőségek

A PRD-ben szereplő korlátok és függőségek azt mutatják meg, hogy a terméktervezés nem kizárólag a kívánt funkciókról szól, hanem azokról a feltételekről is, amelyek között a projekt megvalósítható. Egy webalkalmazás fejlesztése során a technológiai, időbeli, pénzügyi és csapatméretből adódó korlátok közvetlenül hatnak arra, hogy mi kerülhet be az első verzióba, milyen architektúra választható, és mely kockázatokra kell előre felkészülni.

A WatchDB esetében az egyik legfontosabb korlát a szűkített MVP-fókusz. A projekt célja nem

egy minden funkciót tartalmazó, teljes közösségi platform azonnali megvalósítása, hanem egy olyan első verzió elkészítése, amely bizonyítja a személyes filmnapló alapértékét. Ez a döntés összefügg az időbeli és erőforrásbeli korlátokkal is: a PRD az MVP megvalósítását 6-8 hetes időkeretben képzelel el, kisebb fejlesztői csapattal és alacsony költségvetéssel. Emiatt a fejlesztésnek elsősorban azokra a funkciókra kell koncentrálnia, amelyek nélkül az alkalmazás nem adna önálló értéket.

Korlát	Jelentése a projektben	Tervezési következmény
API-korlátok	A külső filmadatok a TMDB API-ból származnak, amelynek elérhetősége és használati korlátai befolyásolják a keresést.	Debounce-olt keresés, gyorsítótárazás és külön adatlekérő réteg alkalmazása indokolt.
Költségvetés	A PRD alacsony költségű, elsősorban ingyenes szolgáltatási csomagokra épülő megvalósítással számol.	Olyan technológiák és szolgáltatások választása szükséges, amelyek kis felhasználószám mellett is fenntarthatók.
Csapatméret	A projekt kisebb, 2-6 fős csapattal számol.	A funkciókörnek szűknek, a fejlesztési folyamatnak pedig jól priorizálnak kell maradnia.
Időkeret	Az MVP tervezett elkészítése 6-8 hét, a közösségi funkciók pedig későbbi szakaszba kerülnek.	A közösségi és haladó funkciók nem részei az első verziónak, hanem külön iterációkban jelenhetnek meg.

3.4. táblázat. A WatchDB fő projektkorlátai

A külső szolgáltatásokhoz kapcsolódó függőségek külön figyelmet igényelnek. A WatchDB mű-

kódésének egyik alapja a TMDB API, amely a filmadatok, posztterek, címek és leírások forrásaként jelenik meg. Ez gyorsítja a fejlesztést, mert nem szükséges saját filmadatbázist építeni, ugyanakkor kockázatot is jelent: ha az API elérhetősége változik, lelassul, vagy a használati feltételek módosulnak, az közvetlenül érintheti az alkalmazás keresési és filmrészlet-megjelenítési funkcióit.

A hitelesítés és az adatkezelés szintén külső szolgáltatásokra épülhet. A PRD-ben megjelenő Supabase vagy hasonló autentikációs szolgáltató használata egyszerűsíti a bejelentkezés, regisztráció és munkamenet-kezelés megvalósítását. Ez azonban azzal jár, hogy figyelni kell a szolgáltató ingyenes csomagjának korlátaira, az adatvédelmi beállításokra és arra, hogy a rendszer később is használható maradjon.

Függőség	Kockázat	Lehetséges kezelés
TMDB API	Elérhetőségi probléma, használati feltételek változása vagy API-korlátok elérése.	Filmadatok helyi gyorsítótárazása és az API-hívások külön szolgáltatásrétegbe szervezése.
Autentikációs szolgáltató	Ingyenes csomag korlátai, szolgáltatófüggőség vagy konfigurációs hiba.	Használat monitorozása, jogosultságok ellenőrzése és hordozható adatmodell kialakítása.
Hosting szolgáltató	Sávszélességi, teljesítménybeli vagy deployment korlátok.	Képek optimalizálása, lazy loading alkalmazása és a hosting terhelésének figyelése.

3.5. táblázat. A WatchDB külső függőségei és kapcsolódó kockázatai

A korlátok mellett a PRD előfeltevéseket is tartalmaz. Ilyen például, hogy a felhasználók modern böngészőt és stabil internetkapcsolatot használnak, a TMDB továbbra is elérhető marad a projekt számára, az email és jelszó alapú bejelentkezés elegendő az MVP-hez, illetve az alkalmazás kezdetben angol nyelvű marad. Ezek az előfeltevések nem technikai részleteknek tűnnek, mégis fontosak, mert meghatározzák, milyen helyzetekre kell az első verzióban felkészülni, és mely igények kezelhetők későbbi fejlesztési szakaszban.

A nem funkcionális követelmények szintén szorosan kapcsolódnak a korlátokhoz és függőségek-

hez. A gyors oldalbetöltés, a biztonságos munkamenet-kezelés, a mobilon is használható felület és a skálázhatóság nem különálló elvárások, hanem olyan minőségi szempontok, amelyek a választott technológiákra és külső szolgáltatásokra is hatással vannak. Például a filmkeresés teljesítménye nemcsak a felület kialakításán múlik, hanem az API-hívások kezelésén, a gyorsítótárazáson és az adatlekérések optimalizálásán is.

Összességében a korlátok és függőségek dokumentálása segít abban, hogy a WatchDB fejlesztése reális keretek között maradjon. A PRD ezáltal nemcsak azt határozza meg, hogy mit kell megvalósítani, hanem azt is, hogy milyen feltételek mellett, milyen kockázatokkal és milyen kompromisszumokkal történhet a megvalósítás.

4. fejezet

Fejlesztési folyamat

Az előző fejezet a WatchDB termékkonceptióját, célközönségét, MVP-jét és a fejlesztést befolyásoló korlátokat mutatta be. Ezek a döntések nemcsak azt határozzák meg, hogy milyen alkalmazás készül, hanem azt is, hogy a fejlesztési folyamatot hogyan érdemes megszervezni. Egy több funkcióból álló webalkalmazás esetében különösen fontos, hogy a feladatok ne elszigetelten, hanem követhető, prioritizált és ellenőrizhető folyamat részeként valósuljanak meg.

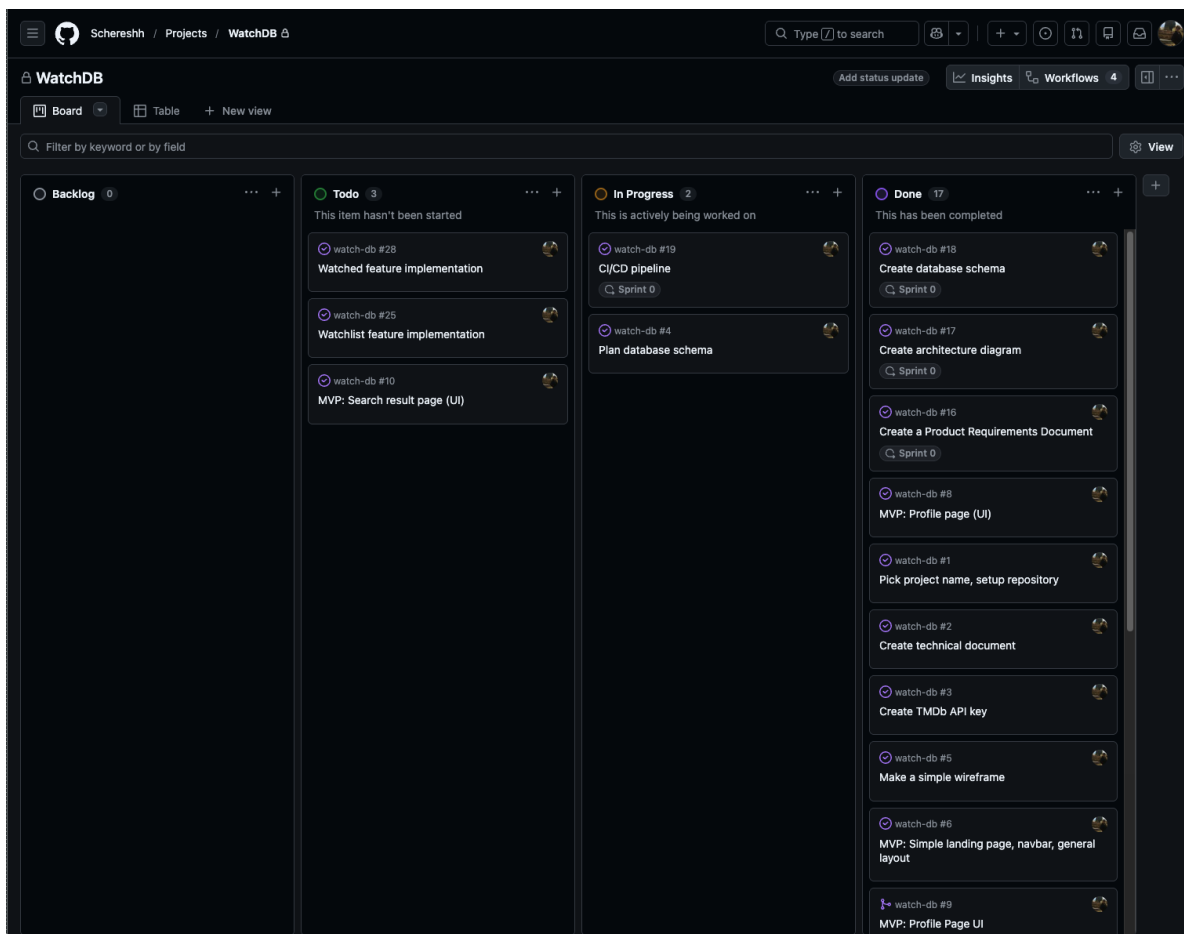
Ez a fejezet röviden bemutatja, hogy a WatchDB fejlesztése milyen projektmenedzsment szemléletre épülhet. A hangsúly a sprintalapú, iteratív munkaszervezésen, a backlog és a feladatok kezelésén, valamint azon a fejlesztési workflow-n van, amely a kiválasztott feladattól a production deploymentig vezeti végig a folyamatot.

4.1. A fejlesztési folyamat kiválasztása

A WatchDB fejlesztési folyamatának kiválasztásánál a projekt mérete, az MVP-re épülő termékstratégia és a későbbi bővíthetőség voltak a legfontosabb szempontok. A korábbi fejezetben bemutatott termékkonceptió alapján az alkalmazás nem egyszerre, teljes funkcionalitással készülne el, hanem először egy szűkített, de használható MVP formájában, majd későbbi verziókban közösségi és haladó funkciókkal bővülne. Ez a felépítés természetesen illeszkedik az iteratív fejlesztési szemlélethez, ahol a csapat kisebb lépésekben halad, és az egyes verziók után visszajelzések alapján pontosíthatja a következő feladatokat [39].

A projekt számára ezért egy sprintalapú, de egyszerűsített fejlesztési folyamat tekinthető megfelelőnek. A Scrumhoz hasonlóan a munka időben elkülönített iterációkra bontható, ahol minden sprintnek van egy előre meghatározott célja, például az autentikáció, a filmkeresés, a watchlist vagy a profilnézet elkészítése [54]. Ugyanakkor egy kisebb projekt esetében nem szükséges minden Scrum-eseményt és szerepkört teljes formalitással bevezetni. A hangsúly inkább azon van, hogy a feladatok átláthatóan legyenek prioritizálva, a haladás követhető legyen, és minden iteráció végén működő, ellenőrizhető eredmény álljon rendelkezésre.

A fejlesztési folyamat gyakorlati alapját a backlog vagy GitHub Issues jellegű feladatkezelés adhatja. Ide kerülnek a PRD-ből levezetett funkciók, a hibajegyek, a technikai feladatok és a későbbi fejlesztési ötletek. A feladatok prioritizálása az MVP logikáját követi: először azok az elemek kerülnek előtérbe, amelyek nélkül az alkalmazás nem nyújtana önálló értéket, míg a közösségi és kényelmi funkciók későbbi iterációkba sorolhatók.



4.1. ábra. A WatchDB fejlesztési feladatainak követése GitHub Projects felületen

Ez a megközelítés azért előnyös, mert összekapcsolja a terméktervezést és a megvalósítást. A PRD nem különálló dokumentumként marad meg, hanem közvetlenül befolyásolja a backlogot, a sprintek céljait és a fejlesztési sorrendet. Így a projektmenedzsment nem pusztán adminisztratív réteggént jelenik meg, hanem a fejlesztés irányításának gyakorlati eszközeként.

4.2. Sprintalapú működés

A sprintalapú működés célja, hogy a fejlesztés ne egyetlen nagy, nehezen követhető feladatként jelenjen meg, hanem kisebb, jól ellenőrizhető iterációkra bontva haladjon. A WatchDB esetében ez azért különösen hasznos, mert az MVP és a későbbi verziók már a PRD-ben is elkülönülnek egymás-

tól. Így a sprintek nem önkényesen kijelölt időszakok, hanem a terméktervezés során meghatározott prioritások gyakorlati megvalósításai.

A folyamat első lépése egy előkészítő sprint, vagyis a Sprint 0 lehet. Ebben nem feltétlenül készül még látványos felhasználói funkció, viszont létrejönnek azok az alapok, amelyekre a későbbi fejlesztés épülhet: a projektstruktúra, az adatbázis kezdeti terve, az autentikációs irány, a CI/CD alapjai, valamint a kezdeti backlog. Ez a szakasz azért fontos, mert csökkenti annak kockázatát, hogy a fejlesztés később szervezetlen technikai döntésekre épüljön.

Az MVP-hez kapcsolódó sprintek már konkrét felhasználói értéket adó funkciókra koncentrálnak. Ilyen lehet például a regisztráció és bejelentkezés elkészítése, a filmkeresés és filmrészletek megjelenítése, a watchlist kezelése, a megnézett filmek naplózása, az értékelés, valamint a saját profiloldal kialakítása. Ezek a funkciók egymásra épülnek, ezért a sprinttervezésnél fontos, hogy a technikai függőségek és a felhasználói folyamatok sorrendje is figyelembe legyen véve.

A későbbi sprintek a termék közösségi és haladó funkcióira épülhetnek. A PRD alapján a v1.1 verzióban jelenhetnek meg a publikus profilok, a követési rendszer, az aktivitási feed és a reakciók, míg a v1.2 már mélyebb funkciókat, például egyéni listákat, statisztikákat, kommenteket vagy ajánlásokat tartalmazhat. Ezek nem tartoznak az MVP alapvető működéséhez, de a sprintalapú tervezés lehetővé teszi, hogy később kontrollált módon kerüljenek be a fejlesztési folyamatba.

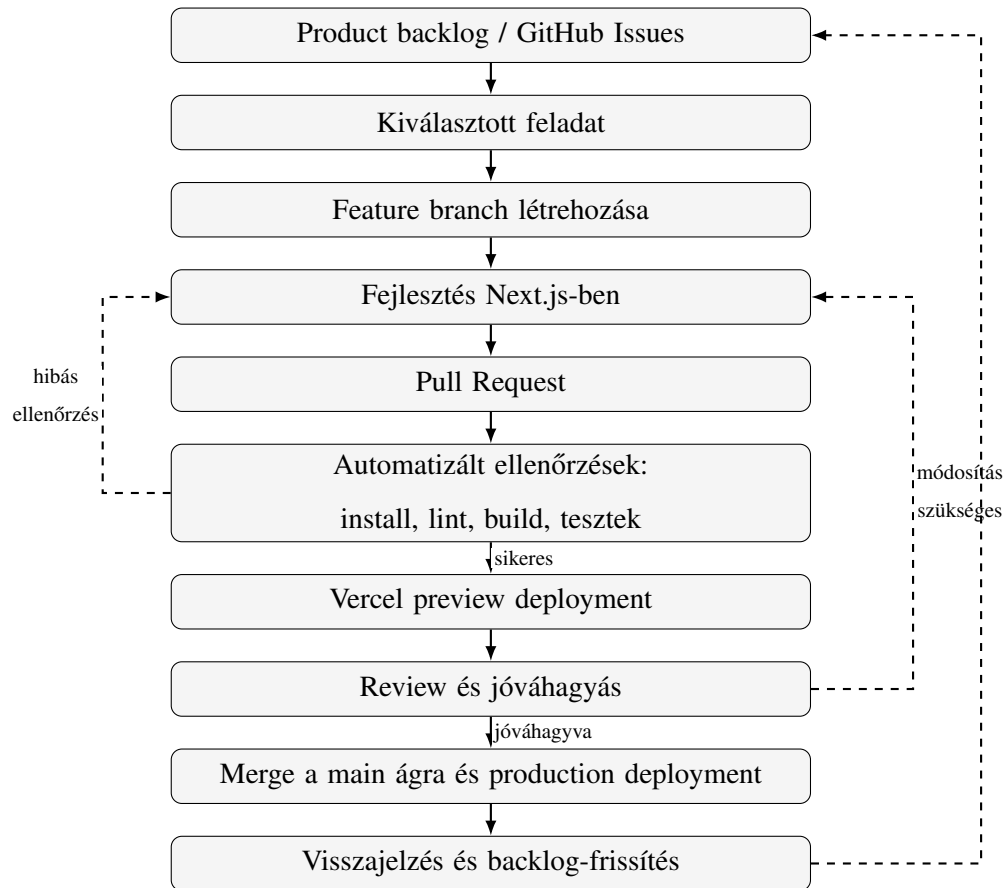
Minden sprint végén fontos valamilyen ellenőrizhető eredmény. Ez lehet egy működő feature, egy javított folyamat, egy sikeres build vagy egy tesztelhető preview deployment. A sprint lezárása után a visszajelzések alapján frissíthető a backlog, módosíthatók a prioritások, és pontosítható a következő sprint célja. Így a fejlesztési folyamat nem merev tervként, hanem folyamatosan finomítható munkamenetként működik.

4.3. Feladatkezelés és fejlesztési workflow

A sprintalapú működés gyakorlati alapját a feladatok egyértelmű nyilvántartása adja. A WatchDB esetében a PRD-ből levezetett funkciók, hibajegyek és technikai teendők GitHub Issues formájában jelennek meg. Egy feladat akkor kezelhető jól, ha röviden leírja a megoldandó problémát, kapcsolódik egy termékcélhoz vagy user flow-hoz, és tartalmaz olyan elfogadási feltételeket (acceptance criteria),

amelyek alapján a munka később ellenőrizhető.

A fejlesztési workflow célja, hogy minden változtatás követhető és ellenőrizhető úton jusson el a felhasználókhoz. A kiválasztott feladatból feature branch készül, ezen történik az implementáció, majd a módosítás Pull Request formájában kerül visszaellenőrzésre. A Pull Request nemcsak technikai lépés, hanem minőségbiztosítási pont is: itt ellenőrizhető, hogy a módosítás illeszkedik-e a termékcélokhoz, nem sérti-e a meglévő működést, és megfelel-e a projekt kódolási elvárásainak.



4.2. ábra. A WatchDB fejlesztési workflow-ja

A CI/CD folyamat szerepe, hogy a Pull Request után automatikusan lefusson néhány alapvető ellenőrzés, például a függőségek telepítése, a lintelés, a buildelés és a tesztek futtatása. Ez csökkenti annak esélyét, hogy hibás vagy nem buildelhető kód kerüljön a fő ágba. Ha az ellenőrzések hibát jeleznek, a feladat visszakerül a fejlesztési szakaszba; sikeres ellenőrzés esetén preview deployment készülhet, amelyen a módosítás még a production környezet előtt kipróbálható [57].

A preview környezet különösen hasznos webalkalmazásoknál, mert nemcsak a kód, hanem a felhasználói élmény is ellenőrizhető. A reviewer itt végigmehet az érintett user flow-n, például megnézheti, hogy egy film hozzáadható-e a watchlisthez, vagy a naplózott film megjelenik-e a profiloldalon. Ez a folyamat segít abban, hogy a fejlesztés ne csak technikailag legyen sikeres, hanem termékoldali szempontból is megfeleljen az elvárásoknak.

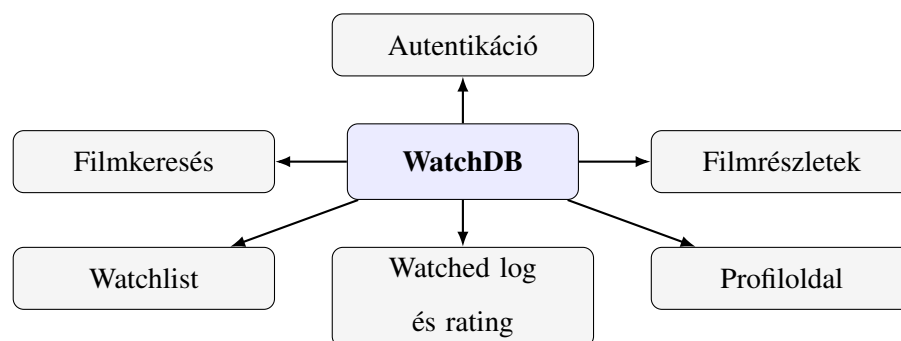
A merge után a változtatás a main ágra kerül, ahonnan production deployment készülhet. A folyamat azonban nem zárul le a telepítéssel: a használatból, review-ból vagy tesztelésből származó visszajelzések újra bekerülhetnek a backlogba. Ez biztosítja, hogy a fejlesztés iteratív maradjon, és a későbbi sprintek valódi tapasztalatokra épüljenek.

5. fejezet

Rendszer specifikációk

5.1. Funkcionális követelmények

A WatchDB MVP verziójának fő funkcionális egységei a következők:

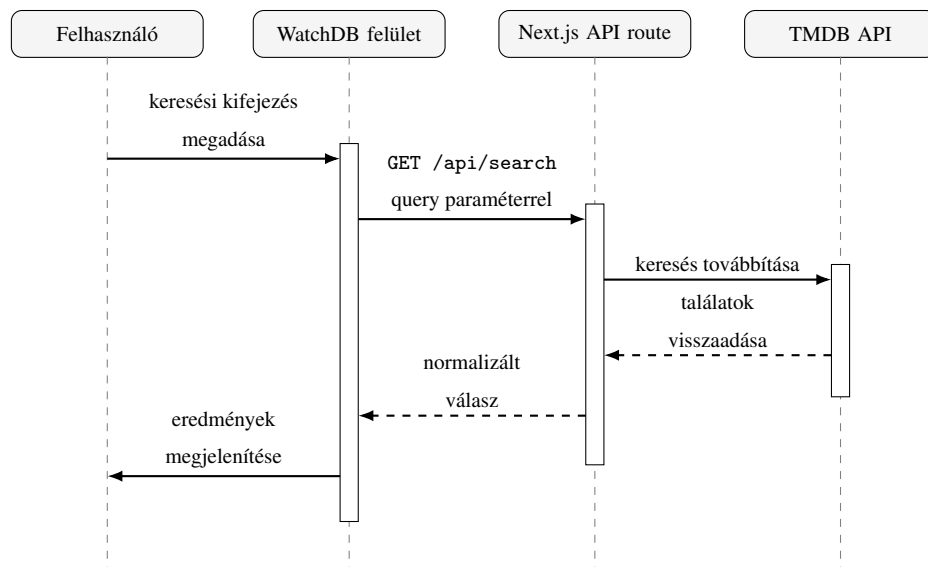


5.1. ábra. A WatchDB fő funkcionális egységei

- A rendszernek biztosítania kell az új felhasználók regisztrációját, a meglévő felhasználók bejelentkezését és kijelentkezését.
- A rendszernek kezelnie kell a felhasználói munkamenetet, hogy a személyes funkciók csak bejelentkezett állapotban legyenek elérhetők.
- A látogatónak tudnia kell cím alapján filmeket keresni.

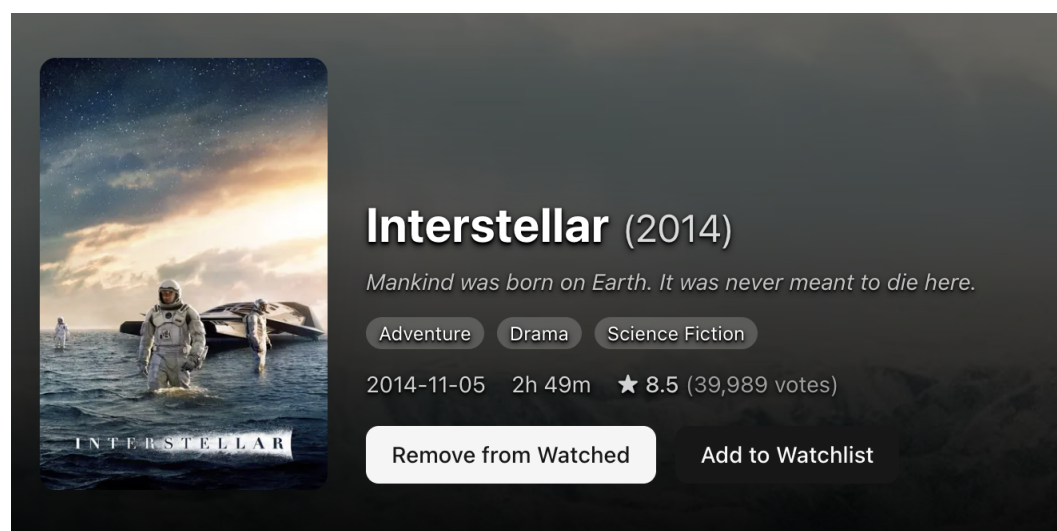
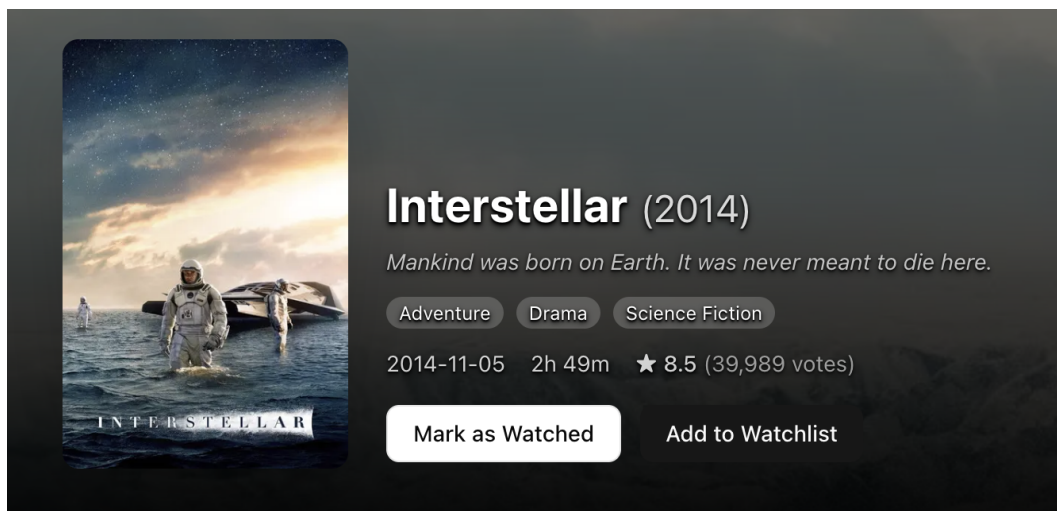
- A keresési eredmények között meg kell jelenjen a film címe, posztere, megjelenési éve és rövid leírása.
- A felhasználónak a keresési találatokból tovább kell tudnia lépni az adott film részletes oldalára.
- A filmrészletek oldalon meg kell jelenjenek a film fontosabb adatai, például cím, poszter, háttérkép, megjelenési dátum, játékidő, műfajok, leírás és értékelési információ.
- A bejelentkezett felhasználónak hozzá kell tudnia adni egy filmet a watchlisthez.
- A bejelentkezett felhasználónak el kell tudnia távolítani egy filmet a watchlistből.
- A rendszernek jeleznie kell, ha egy film már szerepel a felhasználó watchlistjén.
- A bejelentkezett felhasználónak meg kell tudnia jelölni egy filmet megnézettként.
- A megnézettként jelölés során a felhasználónak személyes értékelést kell tudnia megadni.
- Ha egy film megnézettként kerül naplózásra, és korábban szerepelt a watchlisten, a rendszernek el kell távolítania onnan.
- A profiloldalon meg kell jelenjen a felhasználó watchlistje és a megnézett filmek listája.
- A megnézett filmeknél meg kell jelenjen a személyes értékelés és a naplózás vagy megtekintés dátuma.

A filmkeresés fő adatáramlását az alábbi ábra szemlélteti:



5.2. ábra. A filmkeresés fő adatáramlása

A megnézettként jelölés állapotváltozása a filmrészletek oldalon a következőképpen jelenik meg:



5.3. ábra. A megnézettként jelölés állapotváltozása a filmrészletek oldalon

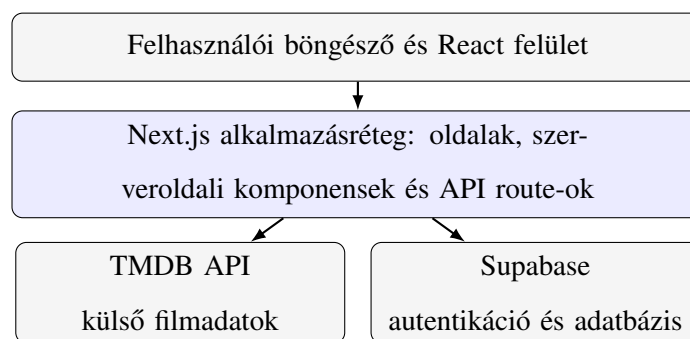
5.2. Nem funkcionális követelmények

- Az alkalmazás webes felületen érhető el, ezért a használatához modern, JavaScript futtatását támogató böngésző szükséges.
- A filmek kereséséhez, a filmrészletek betöltéséhez, valamint a felhasználói listák kezeléséhez folyamatos internetkapcsolat szükséges.
- A filmek böngészése és a filmrészletek megtekintése regisztráció nélkül is elérhető.

- A watchlist, a megnézett filmek listája, az értékelés és a profiloldal használatához regisztrált felhasználói fiók szükséges.
- A regisztráció e-mail cím és jelszó megadásával történik, a fiók aktiválásához pedig az e-mail cím megerősítése szükséges.
- A bejelentkezett állapot fenntartásához a böngészőnek kezelnie kell a munkamenethez szükséges sütiket.
- A filmadatok külső filmes adatforrásból érkeznek, ezért a rendszer működéséhez a TMDB API elérhetősége és megfelelő szervertoldali API-kulcs szükséges.
- A felhasználói adatok tárolása Supabase adatbázisban történik, ezért az autentikációhoz és a személyes listák kezeléséhez a Supabase szolgáltatás elérhetősége szükséges.
- Az alkalmazás használatához nincs szükség külön telepített kliensprogramra vagy speciális hardverre; elegendő egy internetkapcsolattal rendelkező eszköz és egy böngésző.

5.3. Rendszerarchitektúra és technológiai stack

A WatchDB egy webalkalmazásként épül fel, amelyben a felhasználói felület, az alkalmazáslogika, a külső filmadat-forrás és a felhasználói adatok tárolása elkülönített szerepet kap.



5.4. ábra. A WatchDB magas szintű rendszerarchitektúrája

- Az alkalmazás alapját a Next.js App Router adja, amely az oldalak, layoutok, szervertoldali komponensek és API route-ok szervezésére szolgál.

- A felhasználói felület React komponensekből épül fel, külön kliensoldali komponensekkel azoknál a részeknél, ahol interakcióra, állapotkezelésre vagy felhasználói műveletre van szükség.
- A rendszer TypeScriptet használ, amely segíti a komponensek, API-válaszok és szolgáltatásrétegek típusbiztosabb kezelését.
- Az autentikációt és a felhasználóhoz kötött adatok tárolását a Supabase biztosítja. Ide tartozik például a watchlist, a megnézett filmek listája, az értékelés és a megtekintési dátum.
- A filmek adatai nem saját adatbázisból származnak, hanem egy külső a TMDB (The Movie Database) által létrehozott API-ból. Az alkalmazás elsősorban a felhasználói állapotokat tárolja, a filmmel kapcsolatos adatokat pedig külső forrásból kéri le.
- A kliensoldali adatlekérések és állapotfrissítések kezelését a TanStack Query teszi lehetővé, például a watchlist és watched állapotok lekérdezését, illetve azok módosítását.
- A felület kialakításához Tailwind CSS, Radix UI alapú komponensek, valamint shadcn/ui jellegű komponensstruktúra használható.
- Az alkalmazás telepítése és futtatása Vercel környezetben illeszkedik a Next.js alapú felépítéshez; a külső szolgáltatások eléréséhez környezeti változóknál tárolt Supabase és TMDB konfiguráció szükséges.

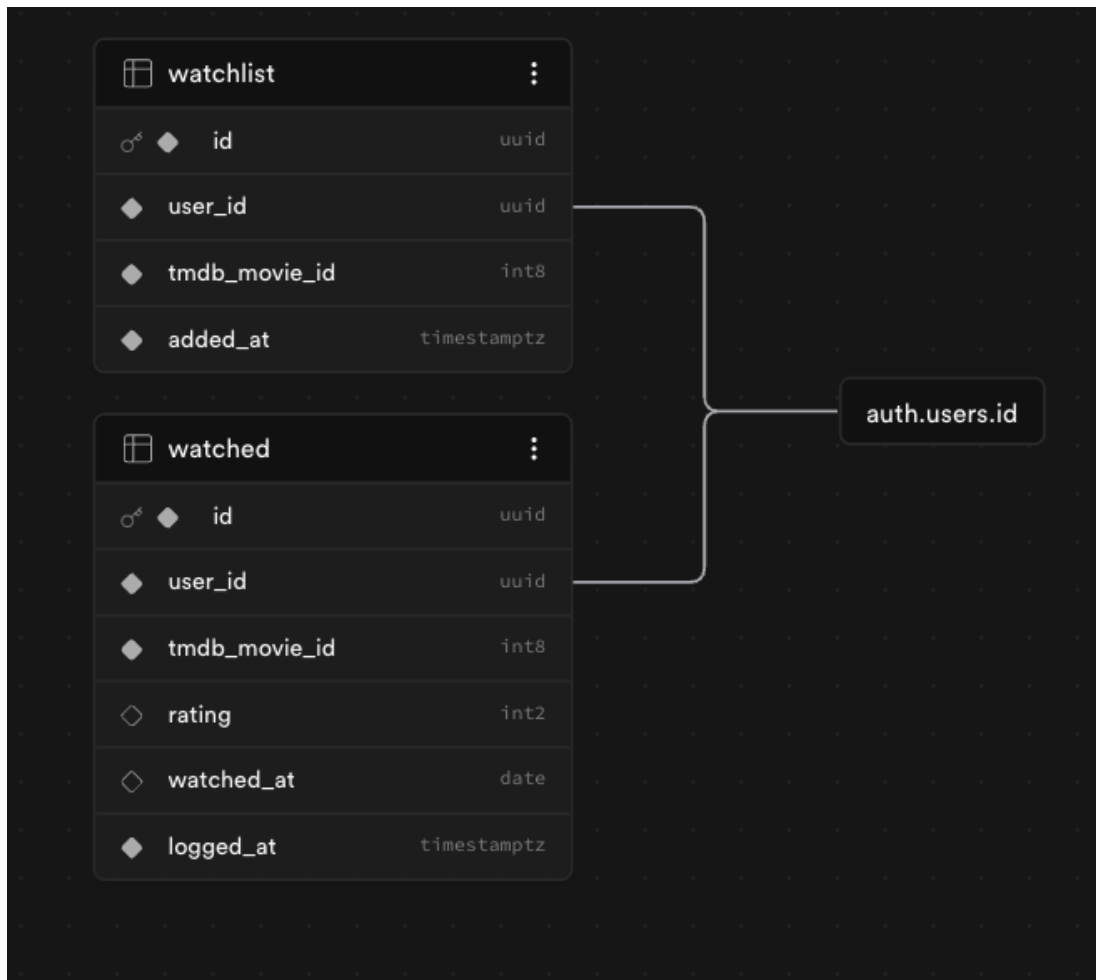
5.4. Adatmodellek és fő entitások

A WatchDB jelenlegi MVP verziójának adatmodellje egyszerű, mert a rendszer nem saját filmadatbázist épít. A filmek részletes adatai külső forrásból, a TMDB API-ból érkeznek, az alkalmazás saját adatbázisa pedig elsősorban a felhasználóhoz kötött állapotokat tárolja. Ennek megfelelően a legfontosabb entitások a felhasználó, a watchlist elem, a megnézett film bejegyzés és a külső film entitás.

A felhasználó és a watchlist között egy-a-több kapcsolat van: egy felhasználó több filmet is elmenthet későbbi megtekintésre. Egy adott film egy felhasználó watchlistjén csak egyszer szerepelhet, amit a `user_id` és a `tmdb_movie_id` párosa azonosít.

A megnézett filmek külön watched bejegyzésként jelennek meg. A rating mező a felhasználó személyes értékelését tárolja 1 és 5 közötti értéként. A watched_at opcionálisan azt jelöli, hogy a felhasználó mikor nézte meg a filmet, míg a logged_at a naplózás időpontját rögzíti. Amikor egy film megnézettként kerül mentésre, a rendszer a hozzá tartozó watchlist bejegyzést eltávolítja.

Fontos tervezési döntés, hogy a WatchDB nem tárolja el a filmek teljes metaadatait saját adatbázisban. A tmdb_movie_id mező logikai kapcsolatot teremt a felhasználói adatok és a TMDb filmadatok között. Ennek előnye, hogy az adatmodell egyszerű marad, és nem szükséges saját filmkatalógust karbantartani. Hátránya, hogy a filmcímek, posztterek és egyéb részletek megjelenítése a külső API elérhetőségétől függ.



5.5. ábra. A WatchDB adatbázissémája

A külső filmdatok a TMDB API-ból érkeznek. Egy részletes filmlekérés eredménye többek között a film azonosítóját, címét, leírását, poszter- és háttérképének elérési útját, megjelenési dátumát, játékidjét, értékelési adatait, státuszát, eredeti nyelvét és műfajait tartalmazza. A külső API válasza a TMDB mezőneveit követi, például `poster_path`, `backdrop_path`, `release_date`, `vote_average`, `vote_count` és `original_language`. Az alkalmazás ezt nem változatlan formában adja tovább, hanem egy belső, JavaScript környezetben természetesebb, camelCase mezőneveket használó objektummá alakítja.

```

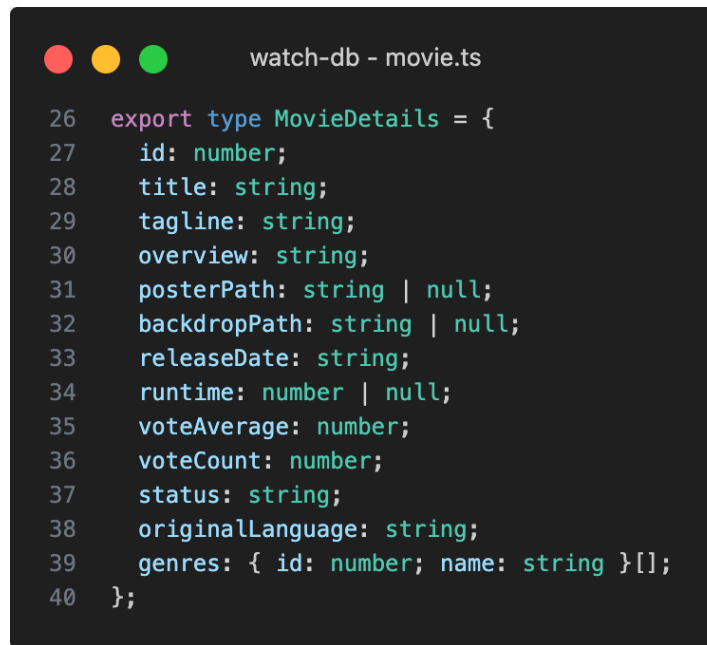
watch-db - movie.ts

42 const getCachedMovieDetails = unstable_cache(
43   async (movieId: number): Promise<MovieDetails> => {
44     const accessToken = process.env.TMDB_ACCESS_TOKEN;
45     if (!accessToken) {
46       throw new Error("TMDB_ACCESS_TOKEN is not set");
47     }
48
49     const client = createTmdbClient({ accessToken });
50
51     const raw = await client.request<TmdbMovieDetails>(`/movie/${movieId}`, {
52       language: "en-US",
53     });
54
55     return {
56       id: raw.id,
57       title: raw.title,
58       tagline: raw.tagline,
59       overview: raw.overview,
60       posterPath: raw.poster_path,
61       backdropPath: raw.backdrop_path,
62       releaseDate: raw.release_date,
63       runtime: raw.runtime,
64       voteAverage: raw.vote_average,
65       voteCount: raw.vote_count,
66       status: raw.status,
67       originalLanguage: raw.original_language,
68       genres: raw.genres,
69     };
70   },
71   ["tmdb-movie-details"],
72   {
73     revalidate: 60 * 60 * 24,
74   },
75 );

```

5.6. ábra. A TMDB filmdatok lekérése és normalizálása

A kódrészletben az alkalmazás lekéri a film részletes adatait a TMDB API-ból, majd a kapott nyers választ egy belső `MovieDetails` objektummá alakítja. Ennek során a külső API `snake_case` mezőnevei egységes `camelCase` mezőnevekké válnak, például a `poster_path` mezőből `posterPath`, a `release_date` mezőből pedig `releaseDate` lesz.



```
26 export type MovieDetails = {
27   id: number;
28   title: string;
29   tagline: string;
30   overview: string;
31   posterPath: string | null;
32   backdropPath: string | null;
33   releaseDate: string;
34   runtime: number | null;
35   voteAverage: number;
36   voteCount: number;
37   status: string;
38   originalLanguage: string;
39   genres: { id: number; name: string }[];
40 };
```

5.7. ábra. A MovieDetails típusdefiníciója

Ugyanez az elv jelenik meg a keresési találatoknál is, de ott az alkalmazás szűkebb adatmodellt használ. A keresési lista csak azokat a mezőket tartja meg, amelyek a találati listához szükségesek: az azonosítót, címet, leírást, posztert, megjelenési dátumot és külső értékelési átlagot. A profiloldalon megjelenő watchlist és watched listák ennél is tovább szűkítik az adatot: a TMDB-ből lekért filmcím és poszter mellé csak a saját adatbázisból származó felhasználói állapot kerül, például a hozzáadás ideje, az értékelés vagy a megtekintés dátuma.

5.5. API végpontok és adatáramlások

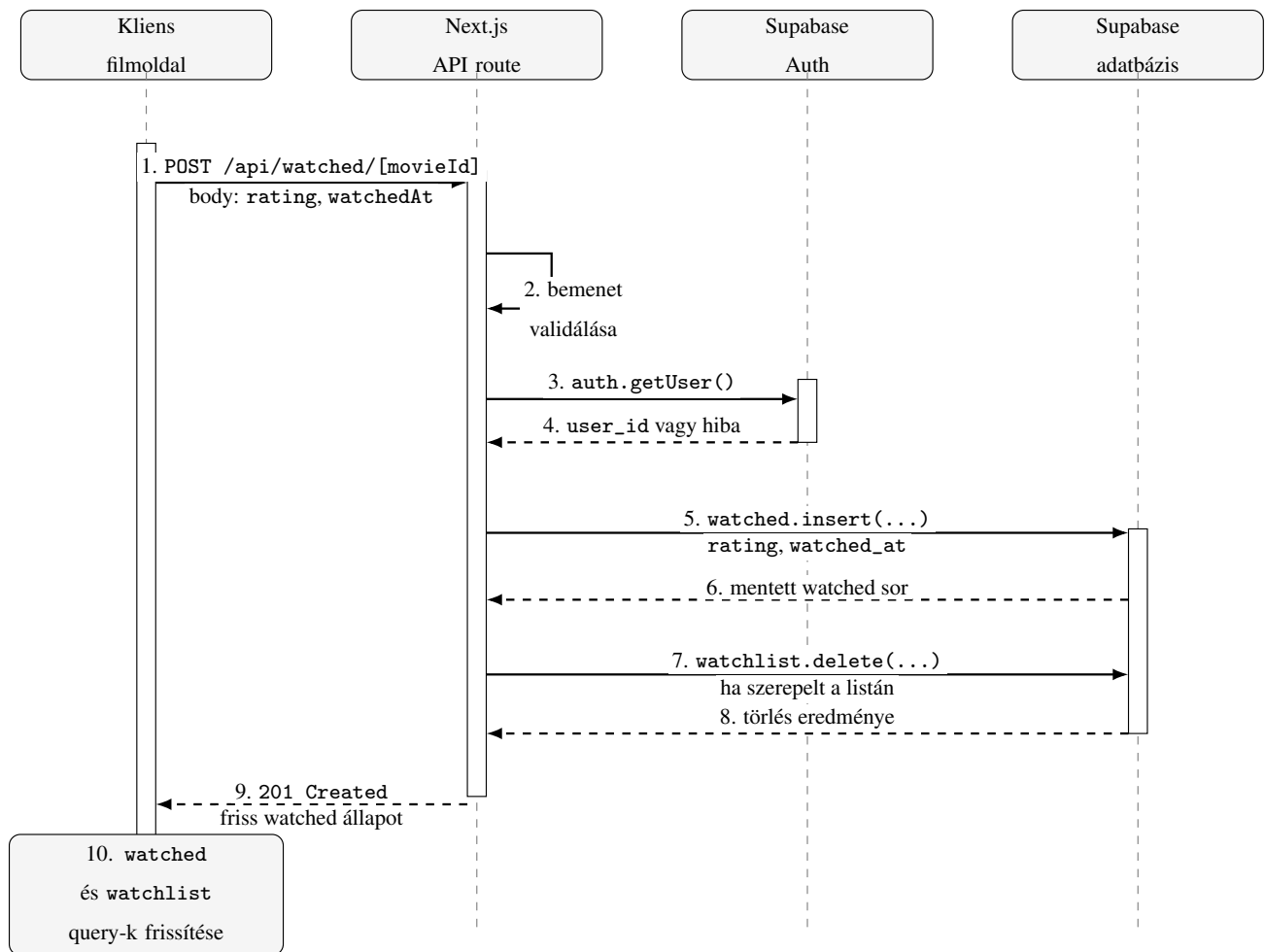
A WatchDB a Next.js API route-jait használja a kliensoldali műveletek kiszolgálására. A böngésző nem közvetlenül kommunikál sem a Supabase adatbázissal, sem a TMDB API-val: ezek elérése szerveroldalon történik. Ez egyrészt védi a külső szolgáltatásokhoz tartozó kulcsokat, másrészt egységesebb hibakezelést és válaszformátumot tesz lehetővé. A sikeres válaszok data mezőben, a hibák pedig error mezőben térnek vissza.

Végpont	Metódus	Paraméterek	Szerep
/api/search	GET	query: kötelező page: opcionális	Filmkeresés a TMDB keresési endpointján keresztül.
/api/watchlist	GET	Nincs query. Felhasználó: sessionből.	A bejelentkezett felhasználó watchlistjének lekérése.
/api/watchlist/ [movieId]	GET POST DELETE	movieId: route paraméter Nincs request body.	Egy film watchlist állapotának lekérése, hozzáadása vagy eltávolítása.
/api/watched	GET	page: opcionális limit: opcionális	A megnézett filmek paginált listájának lekérése.
/api/watched/ [movieId]	GET POST DELETE	movieId: route paraméter POST body: rating POST body: watchedAt	Egy film watched állapotának lekérése, megnézettként jelölése vagy eltávolítása.

5.1. táblázat. A WatchDB fő API végpontjai

A lekérdező végpontok kétféle adatforrással dolgoznak. A keresés és a filmrészletek a TMDB API-ból származó adatokat használják, míg a watchlist és watched listák először a Supabase adatbázisból kéri le a felhasználóhoz tartozó sorokat. A visszakapott tmdb_movie_id értékek alapján az alkalmazás ezután lekéri a szükséges filmadatokat a TMDB-ből, majd a saját adatbázisból és a külső API-ból származó adatokat egy közös válaszobjektumban adja vissza.

Az állapotmódosító műveleteknél a kliensoldalon a TanStack Query kezeli a lekérdezések és mutációk (CREATE, UPDATE, DELETE) állapotát. Egy watchlist vagy watched művelet után az érintett query-k frissülnek, így a filmrészletek oldalon és a profiloldalon megjelenő adatok újratöltés nélkül is követhetik a változást.



5.8. ábra. A megnézettként jelölés adatáramlása

5.6. Autentikáció

A WatchDB autentikációját a Supabase Auth biztosítja. A rendszer külön böngészőoldali és szervertoldali Supabase klienst használ: a böngészőoldali kliens a regisztrációhoz és bejelentkezéshez kapcsolódik, a szervertoldali kliens pedig az API route-okban és a védett oldalak betöltésekor ellenőrzi a felhasználói munkamenetet. Ez illeszkedik a Next.js szervertoldali működéséhez, ahol a session kezelése cookie-kon keresztül történik.

A funkciók többsége (pl. megnézendő / megnézettként jelölés) csak bejelentkezett felhasználóhoz köthető. A profiloldal például szervertoldalon ellenőrzi, hogy létezik-e aktív felhasználó, ennek hiá-

nyában a rendszer a bejelentkező oldalra irányítja a látogatót. A watchlist és watched műveleteknél az API route-ok szintén a Supabase Auth által visszaadott felhasználó alapján dolgoznak. Így a rendszer nem a kliens által küldött felhasználóazonosítóra támaszkodik, hanem a szerveroldalon ellenőrzött sessionból származó `user_id` értékre.

A belépéshez és a munkamenet kezeléséhez a projekt a Supabase kliens inicializálásához szükséges környezeti változókat használja:

- `NEXT_PUBLIC_SUPABASE_URL`
- `NEXT_PUBLIC_SUPABASE_ANON_KEY`

A változók önmagukban nem jelentenek jogosultságot az adatokhoz, csak a Supabase projekt elérését teszik lehetővé. A tényleges felhasználói műveletek előtt a szerveroldali réteg minden esetben az aktuális session alapján azonosítja a felhasználót. Ennek köszönhetően a személyes listák és értékelések nem választhatók el a bejelentkezett fióktól.

5.7. Külső szolgáltatások és függőségek

A WatchDB nem teljesen önálló rendszerként működik, hanem több külső szolgáltatásra és fejlesztői csomagra épül. Ezek a függőségek gyorsítják a fejlesztést, mert nem szükséges saját teljes filmadatbázist vagy egyedi UI komponenseket kialakítani. Ugyanakkor a választott szolgáltatások elérhetősége, verziói és használati feltételei közvetlenül hatnak az alkalmazás működésére.

A legfontosabb külső szolgáltatások és függőségek a következők:

- **Supabase:** a felhasználóhoz kötött adatok tárolását biztosítja. A projektben a watchlist és watched táblák kapcsolódnak hozzá. Szolgáltatáskimaradás vagy hibás konfiguráció esetén a személyes funkciók nem érhetők el.
- **TMDB API:** külső filmadatforrásként működik. A filmkeresés, a filmrészletek, a posztterek és háttérképek lekérése ezen keresztül történik [73]. API-korlát, lassulás vagy elérhetetlenség esetén a keresés és a filmoldalak működésének minősége gyengülhet.

- **Vercel / Next.js környezet:** az alkalmazás futtatását, az oldalak kiszolgálását és a szerveroldali route-ok kezelését támogatja. Build- vagy deployment-hiba esetén az új verzió nem kerül élesítésre.
- **TanStack Query:** a kliensoldali serverállapot-kezelésért felel. A watchlist és watched állapotok lekérése, a mutációk (CREATE, UPDATE, DELETE) kezelése és a cache-frissítés is ehhez kapcsolódik. Hibás invalidálás esetén a felület ideiglenesen elavult adatot mutathat, bár ez a hiba inkább fejlesztői oldalról történhet meg, nem pedig egy külső szolgáltatástól függ.
- **UI függőségek:** a felületi komponensek és stílusok kialakítását segítik. Ide tartozik a Tailwind CSS, a Radix UI, a shadcn/ui jellegű komponensek és az ikonok. Verzióváltáskor megjelenési vagy komponens-kompatibilitási problémák jelentkezhetnek.

A felsorolt külső szolgáltatások és a velük járó kockázatok mind számításba lettek véve a fejlesztési folyamatot megelőző tervezési szakaszban.

A külső filmadatok eléréséhez a projekt a `TMDB_ACCESS_TOKEN` környezeti változót használja. Ez csak szerveroldalon kerül felhasználásra, ezért a böngésző közvetlenül nem kap hozzáférést a tokenhez. Ez a döntés biztonsági szempontból fontos, mert így a külső API-hoz tartozó hozzáférési adat nem jelenik meg a kliensoldali kódban.

A fejlesztői csomagok szintén fontos függőséget jelentenek. A `package.json` alapján a projekt fő futásidejű csomagjai közé tartozik a `next`, a `react`, a `@supabase/ssr`, a `@supabase/supabase-js`, a `@tanstack/react-query`, valamint a felülethez kapcsolódó Radix UI, Tailwind és ikoncsomagok. Ezek verzióit kontrolláltan kell frissíteni, mert egy nagyobb keretrendszer- vagy könyvtárfrissítés érintheti az útvonalkezelést, a szerveroldali működést, az adatkapcsolatot vagy a komponensek viselkedését, esetleges biztonsági patcheket is tartalmazhatnak az új verziók.



```
watch-db - package.json
11  "dependencies": {
12    "@radix-ui/react-slot": "^1.2.4",
13    "@radix-ui/react-tabs": "^1.1.13",
14    "@supabase/ssr": "^0.8.0",
15    "@supabase/supabase-js": "^2.93.0",
16    "@tanstack/react-query": "^5.100.4",
17    "class-variance-authority": "^0.7.1",
18    "clsx": "^2.1.1",
19    "lucide-react": "^0.545.0",
20    "next": "15.5.9",
21    "radix-ui": "^1.4.3",
22    "react": "19.1.0",
23    "react-dom": "19.1.0",
24    "react-hook-form": "^7.71.1",
25    "tailwind-merge": "^3.3.1"
26  },
27  "devDependencies": {
28    "@eslint/eslintrc": "^3",
29    "@tailwindcss/postcss": "^4",
30    "@types/node": "^20",
31    "@types/react": "^19",
32    "@types/react-dom": "^19",
33    "eslint": "^9",
34    "eslint-config-next": "15.5.5",
35    "tailwindcss": "^4",
36    "tw-animate-css": "^1.4.0",
37    "typescript": "^5"
38  }
```

5.9. ábra. A projekt fő függőségei a `package.json` alapján

A külső függőségek kezelése miatt a projekt elkülöníti a szolgáltatásrétegeket. A TMDB-vel kommunikáló kód külön modulba került. A Supabase kliens inicializálása külön böngészőoldali és szervertoldali fájlban szerepel. Ez a szervezés csökkenti annak kockázatát, hogy egy külső szolgáltatáshoz kapcsolódó változás az alkalmazás több, egymástól távoli részét érintse.

5.8. Deployment és futtatási környezet

A WatchDB futtatása két fő környezetre bontható: lokális fejlesztői környezetre és éles, felhasználók számára elérhető környezetre. A projekt Next.js alapokra épül, ezért az oldalak, a szerveroldali komponensek és az API route-ok ugyanabban az alkalmazásban találhatók (monorepo-szerű felépítés). Ez egyszerűsíti a deployment folyamatot, mert nincs külön frontend és backend alkalmazás, amelyet eltérő módon kellene telepíteni.

A lokális fejlesztés során a projekt a `package.json` fájlban megadott parancsokra támaszkodik:

- `npm run dev`: fejlesztői szerver indítása Turbopack használatával;
- `npm run build`: production build elkészítése;
- `npm run start`: az elkészült production build lokális indítása;
- `npm run lint`: statikus kódelemzés futtatása.



```
watch-db - package.json

5  "scripts": {
6    "dev": "next dev --turbopack",
7    "build": "next build --turbopack",
8    "start": "next start",
9    "lint": "eslint"
10 }
```

5.10. ábra. A projekt futtatási scriptjei a `package.json` alapján

Az éles környezet szempontjából az alkalmazás Vercel jellegű hostingra illeszkedik. A build folyamat során a szolgáltató telepíti a függőségeket, lefuttatja a production buildet, majd az elkészült

alkalmazást elérhetővé teszi a felhasználók számára. A fejlesztési folyamatban ez összekapcsolható a CI/CD szemlélettel: Pull Request esetén preview deployment készülhet, a fő ágra kerülő változtatásból pedig production deployment jöhet létre.

A fejlesztői környezet megfelelő működéséhez a következő konfigurációkra van szükség:

- Node.js és npm alapú futtatási környezet;
- a Supabase projekthez tartozó publikus URL és anon kulcs;
- a TMDB API szerveroldali hozzáférési tokenje;
- modern, JavaScript futtatását támogató böngésző;
- stabil internetkapcsolat a Supabase, a TMDB API és a TMDB képkiszolgáló eléréséhez.

A `next.config.ts` külön engedélyezi a `image.tmdb.org` tartományról érkező képek használatát. Erre azért van szükség, mert a filmposzterek és háttérképek nem az alkalmazás saját tárhelyéről, hanem a TMDB képkiszolgálójáról töltődnek be. Az alkalmazás működéséhez ezért nemcsak a saját hosting környezet, hanem a külső képszolgáltató elérhetősége is szükséges.

Fontos döntés, hogy a TMDB hozzáférési token nem kerül kliensoldalra. A filmadatok lekérése a szerveroldali szolgáltatásrétegen keresztül történik, így a böngésző csak az alkalmazás saját API route-jaival kommunikál. Ez a megoldás egyszerűbbé teszi a kulcsok kezelését, és csökkenti annak kockázatát, hogy külső szolgáltatáshoz tartozó érzékeny konfigurációs adat kiszivároгjon.

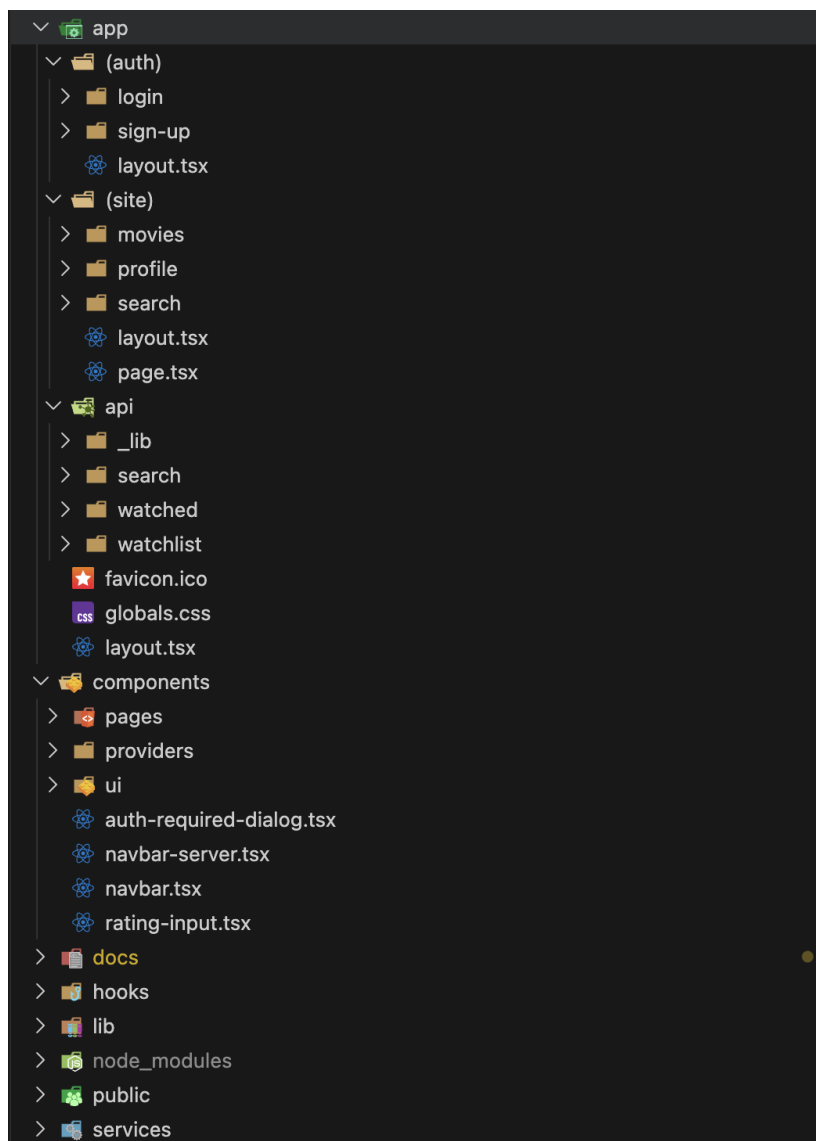
6. fejezet

Megvalósítás

6.1. Megvalósítási megközelítés

A WatchDB megvalósítása a Next.js App Router szervezési logikájára épül. Az alkalmazásban az oldalak, a serveroldali komponensek, a kliensoldali komponensek és az API route-ok egy közös projektstruktúrában találhatók. Ez azért előnyös az MVP szempontjából, mert a frontend felület és a backend jellegű műveletek nem külön alkalmazásként jelennek meg, hanem ugyanazon kódbázison belül kapcsolódnak össze.

A projekt felépítése több rétegre bontható. Az app mappa tartalmazza az útvonalakat, például a keresőoldalt, a film oldalt, a profiloldalt és az API végpontokat. A components mappa a felhasználói felület újrafelhasználható elemeit tartalmazza, például a navigációt, a profiloldali listákat, a dialógusokat és az értékelő komponenst. A hooks mappa a kliensoldali lekérdezésekhez és mutációkhoz kapcsolódik, míg a services mappa a TMDB-hez és a felhasználói listákhoz tartozó adatlekérési logikát különíti el.



6.1. ábra. A WatchDB projekt mappastruktúrája

Ez a felosztás segít abban, hogy a megvalósítás ne egyetlen nagy komponensben koncentrálódjon. A filmadatok lekérése, a felhasználói állapot módosítása és a felület frissítése külön felelősségi körként jelenik meg. Például a film oldal a TMDb-ből érkező adatokat jeleníti meg, míg a watchlist és watched állapotok kezelése API route-okon és TanStack Query alapú kliensoldali hookokon keresztül történik.

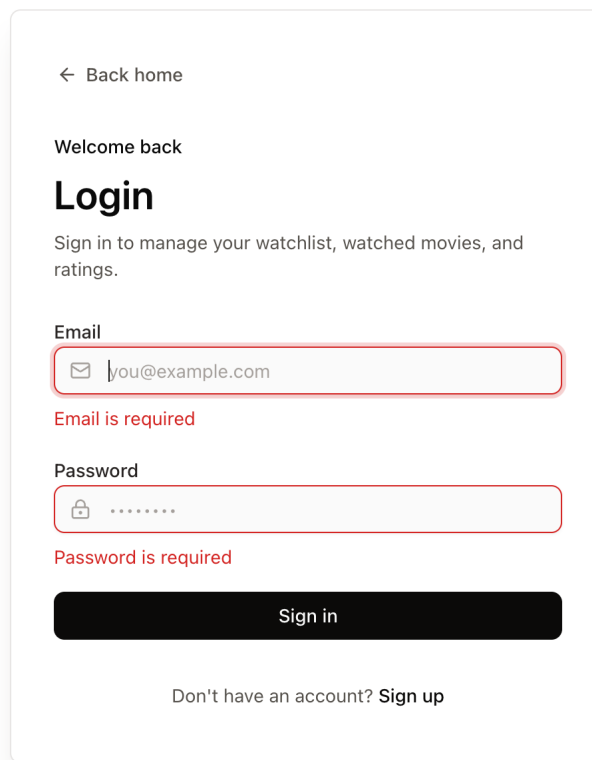
A megvalósítás során fontos szempont volt, hogy a személyes adatok mindig a bejelentkezett felhasználóhoz kapcsolódjanak. Az API route-ok szerveroldalon kérik le az aktuális sessionhöz tartozó felhasználót, és az adatbázis-műveleteket ehhez a fiókhoz kötik. Így például a watchlist bejegyzés, az

értékelés és a megtekintési dátum is a megfelelő felhasználói fiókhoz kerül.

6.2. Autentikáció és védett útvonalak

Az alkalmazás autentikációs része email és jelszó alapú bejelentkezésre épül. A belépéshez és regisztrációhoz külön oldalak tartoznak: a `/login` útvonal a meglévő felhasználók beléptetését, a `/sign-up` útvonal pedig az új felhasználói fiókok létrehozását kezeli.

A bejelentkezési és regisztrációs űrlapok validációját a `react-hook-form` végzi. A login form az email címet és a jelszót ellenőrzi, míg a regisztrációs oldal egy megerősítő jelszómezőt is tartalmaz. Sikeres belépés után az alkalmazás a profiloldalra irányítja a felhasználót, hibás adatbevitel vagy sikertelen autentikáció esetén pedig közvetlenül az űrlapon jelenik meg visszajelzés.



The image shows a login form with the following elements:

- A link: [← Back home](#)
- Greeting: **Welcome back**
- Section title: **Login**
- Description: Sign in to manage your watchlist, watched movies, and ratings.
- Email field: Labeled "Email", contains the text "you@example.com". Below the field is a red error message: "Email is required".
- Password field: Labeled "Password", contains masked characters ".....". Below the field is a red error message: "Password is required".
- Sign in button: A black button with the text "Sign in".
- Footer: "Don't have an account? [Sign up](#)"

6.2. ábra. Hibás bejelentkezési kísérlet

Az autentikációhoz a projekt böngészőoldali és szerveroldali Supabase klienst is használ. A bö-

gészőoldali kliens az űrlapokhoz kapcsolódik, a szerveroldali kliens pedig a védett oldalak és API route-ok esetében kéri le az aktuális felhasználót. A munkamenet cookie-kon keresztül kezelődik, amelynek frissítését a middleware (minden kérés feldolgozása előtt lefutó kód) végzi.

A profiloldal védett útvonalként működik: ha nincs bejelentkezett felhasználó, az alkalmazás a /login oldalra irányítja a látogatót. Aktív session esetén a profiloldal megjeleníti a felhasználóhoz tartozó watched és watchlist listákat.

Ugyanezen az elven működnek az állapotmódosító API route-ok is. A watchlist és watched műveletek előtt a szerveroldali réteg ellenőrzi az aktuális felhasználót, majd az adatbázis műveleteket ehhez a fiókhoz köti.

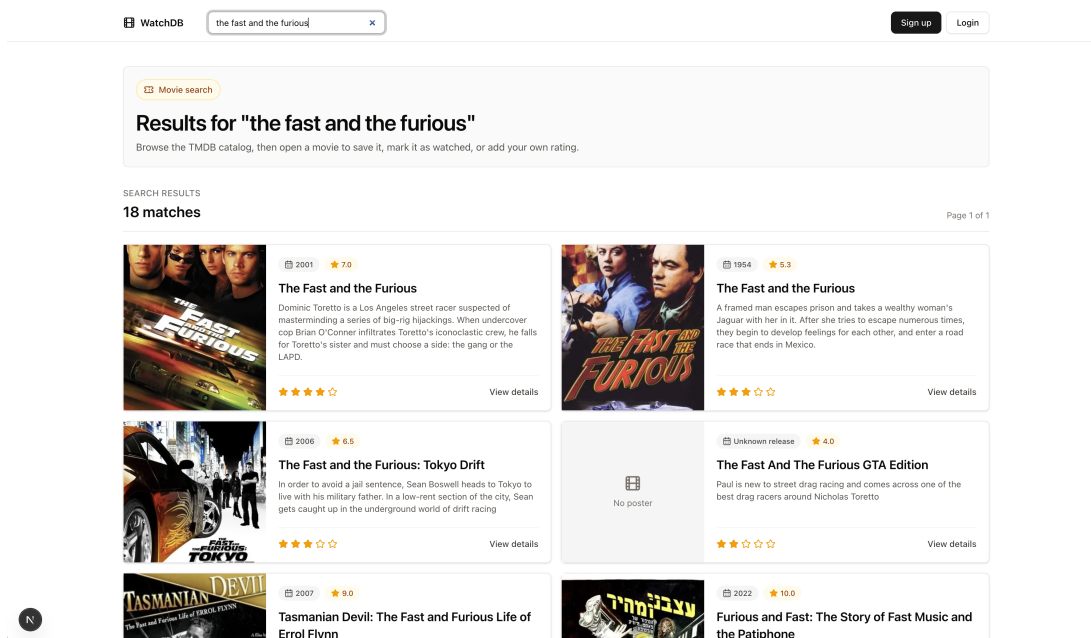
Ez a felépítés egyértelműen elkülöníti a nyilvános és a személyes funkciókat. A filmkeresés és a film oldalak megtekintése bejelentkezés nélkül is használható, míg a watchlist, a megnézett filmek listája, az értékelés és a profiloldal már felhasználói fiókhoz kötött.

6.3. Filmkeresés és film oldal

A filmkeresés az alkalmazás egyik nyilvánosan elérhető funkciója. A keresőmező a navigációs sávban található, így a felhasználó az alkalmazás bármely részéről indíthat keresést. Enter lenyomása-kor a kliensoldali navigáció a /search útvonalra irányít, ahol a keresett kifejezés query paraméterként jelenik meg.

A keresőoldal szerveroldali komponensként működik. Az oldal a searchParams alapján kiolvassa a keresési kifejezést és az aktuális oldalszámot, majd a searchMovies függvényen keresztül lekéri a TMDB találatait. A válaszban érkező adatokat az alkalmazás normalizált formában használja, ezért a felületen már egységes mezőnevek jelennek meg, például posterPath, releaseDate és voteAverage.

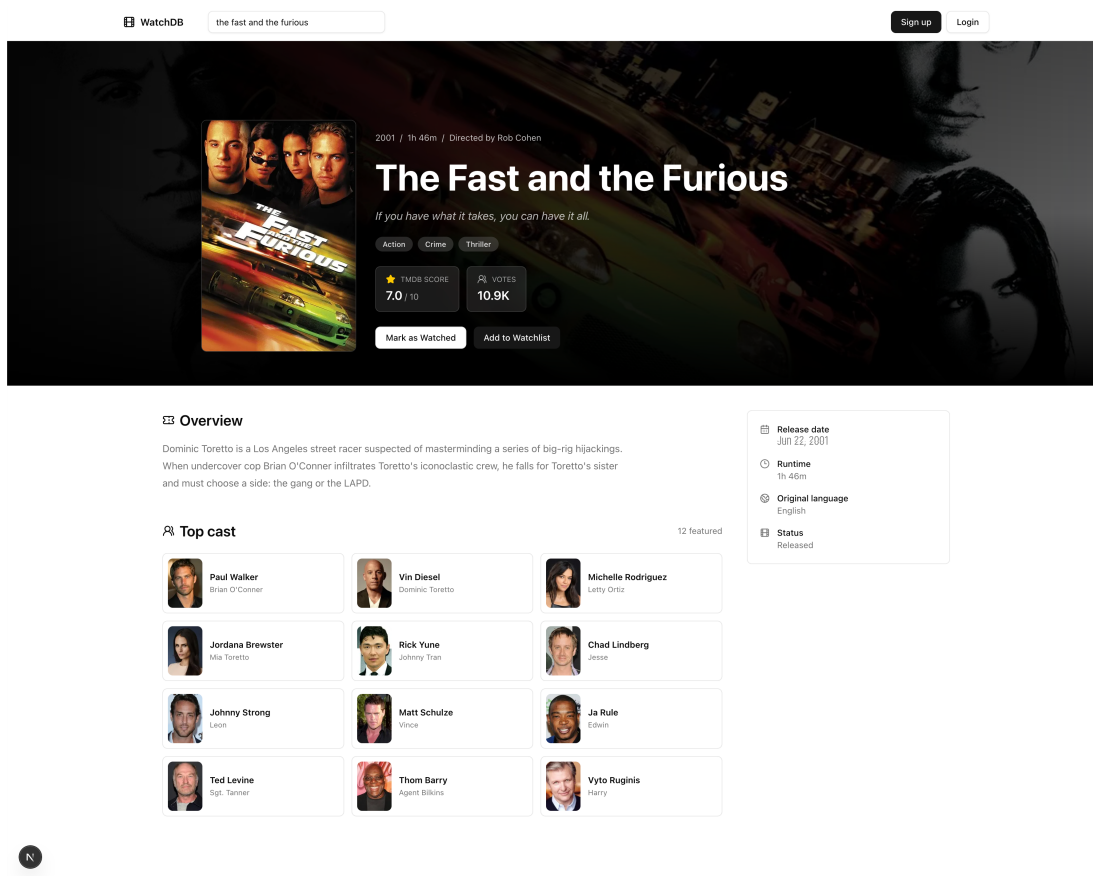
A találatok megjelenítését külön SearchResultCard komponens végzi. A kártya a film plakátját, címét, megjelenési évét, rövid leírását és TMDB értékelését mutatja. Ha egy filmhez nem érhető el plakát vagy leírás, a komponens helyettesítő állapotot jelenít meg, így a felület hiányos külső API-adatok esetén is használható marad. A találati lista lapozását a PaginationButtons komponens kezeli.



6.3. ábra. A filmkeresési találatok megjelenítése

A film oldal a `/movies/[id]` dinamikus útvonalon érhető el. Az oldal először számmá alakítja az URL-ben érkező azonosítót, majd hibás azonosító vagy sikertelen adatlekérés esetén a `Next.js` `notFound` függvényével kezeli a hibás állapotot. Sikeres lekérés esetén a `getMovieDetails` függvény tölti be a filmhez tartozó részletes adatokat.

A film oldal felső része a `MovieDetailHero` komponensben jelenik meg. Ez tartalmazza a háttérképet, a plakátot, a címet, a műfajokat, a rendezőket, a játékidőt és a TMDB értékelést. Ugyanitt találhatóak a műveletek gombjai is, például a watchlisthez adás és a megnézettként jelölés. Az oldal alsó része az áttekintést, a főbb szereplőket és az alapvető metaadatokat jeleníti meg.



6.4. ábra. A film oldal felépítése

6.4. Watchlist, watched és rating funkciók

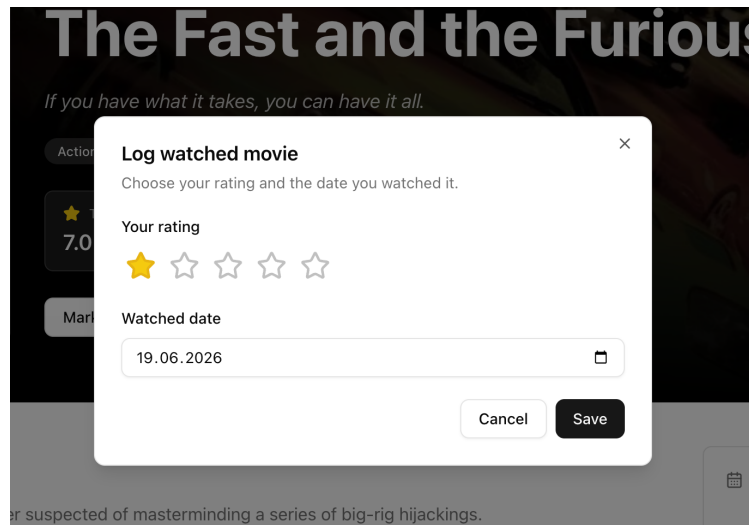
A watchlist, watched és rating funkciók a film oldalon érhetők el. Ezek a műveletek már személyes felhasználói adatokhoz kapcsolódnak, ezért csak bejelentkezett felhasználó esetén hajthatók végre. Ha a látogató nincs bejelentkezve, a gombok nem hajtanak végre műveletet, hanem egy autentikációt kérő dialógust jelenítenek meg.

A watchlist gomb először lekéri, hogy az adott film szerepel-e a felhasználó listájában. Ennek alapján változik a gomb felirata: a film hozzáadható a listához, vagy eltávolítható onnan. A módosítást egy TanStack Query mutáció indítja, amely a watchlist API-végpont POST vagy DELETE metódusát hívja meg.

A watched funkció hasonló felépítésű, de a megnézettként jelölés több adatot kezel. A gomb

először lekéri az aktuális watched állapotot. Ha a film még nincs megnézettként jelölve, a felhasználó egy dialógusban adhatja meg az értékelést és a megtekintés dátumát. Ha a film már szerepel a watched listában, a gomb az eltávolítást indítja el.

Az értékelés megadásához csillagos, egytől ötig terjedő beviteli mező tartozik. A dátummező alapértelmezetten az aktuális napot veszi fel, és nem enged jövőbeli dátumot megadni.



6.5. ábra. A megnézettként jelöléshez tartozó modal

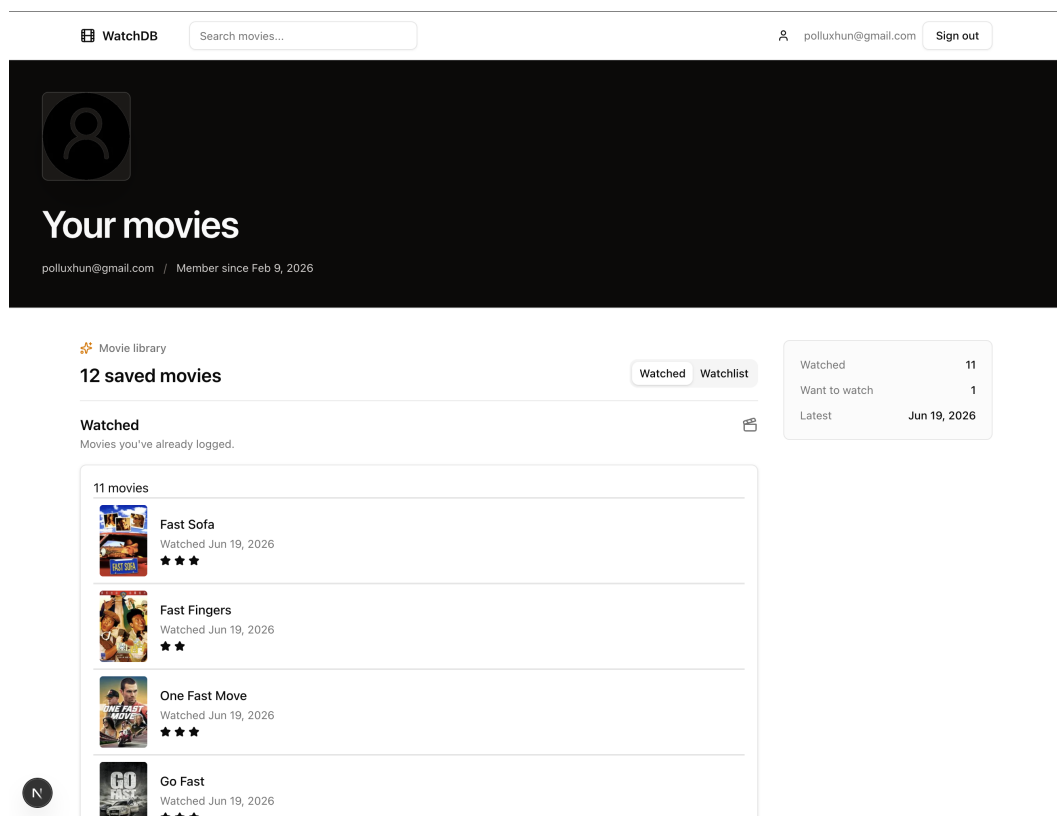
A szerveroldali API route-ok minden módosítás előtt ellenőrzik az aktuális felhasználót. A watched mentésekor a rendszer validálja a film azonosítóját, az értékelést és a dátum formátumát, majd új sort hoz létre a watched táblában. A művelet után a film automatikusan kikerül a watchlist táblából, mert egy már megnézett filmnek nem szükséges továbbra is a megnézendő filmek között szerepelnie.

A kliensoldali élményt a TanStack Query cache-kezelése teszi gyorsabbá. A mutációk optimista frissítést alkalmaznak, vagyis a gombok állapota már a szerverválasz előtt frissül. Hiba esetén a korábbi állapot visszaállítható, sikeres mentés után pedig a watchlist és watched lekérdezések érvénytelenítésre kerülnek. Így a film oldal és a profiloldalon megjelenő listák ugyanazt a friss állapotot tükrözik.

6.5. Profiloldal és kliensoldali adatkezelés

A profiloldal az alkalmazás azon része, ahol a felhasználó a saját filmgyűjteményét követheti. Az oldal csak bejelentkezett felhasználó számára érhető el, ezért a betöltéskor az alkalmazás lekéri a profiladatokat, a megnézett filmeket és a watchlist tartalmát. Ha bármelyik lekérdezés jogosultsági hibát ad vissza, a felhasználó a bejelentkezési oldalra kerül.

A felület felső része a profilhoz tartozó alapinformációkat jeleníti meg, például az e-mail címet és a regisztráció dátumát. A központi tartalom két fülre van bontva: az egyik a megnézett filmeket, a másik a későbbre elmentett filmeket listázza. A jobb oldali összegző rész rövid statisztikákat mutat, például a watched és watchlist elemek számát, valamint a legutóbbi aktivitás dátumát.



6.6. ábra. A profiloldal felhasználói felülete

A kliensoldali adatkezelést a TanStack Query biztosítja. A profiladatok, a watchlist lista és a watched lista külön query kulcsokhoz kapcsolódnak. Ez azért hasznos, mert a film oldalon végrehajtott

módosítások után pontosan azok a cache-bejegyzések frissíthetők vagy érvényteleníthetők, amelyeket a profiloldal is használ.

7. fejezet

Tesztelés és minőségellenőrzés

7.1. A tesztelési stratégia célja

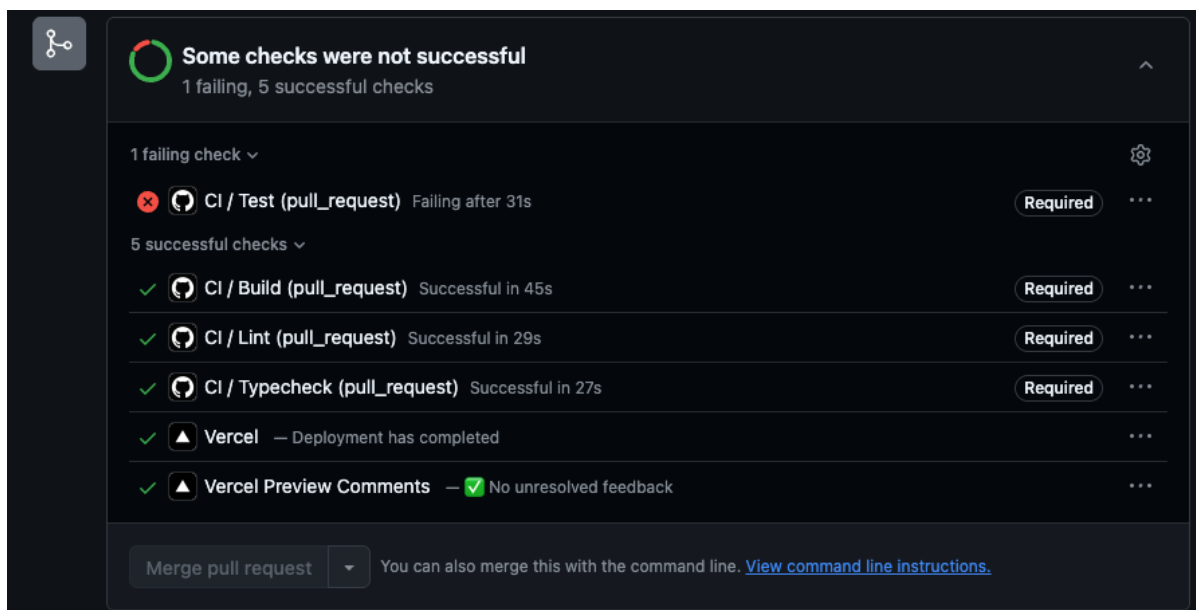
A WatchDB fejlesztése során a tesztelés célja az volt, hogy a rendszer legfontosabb működési pontjai ellenőrizhetőek legyenek, és a későbbi módosítások ne okozzanak észrevétlen regressziós hibákat. Egy webalkalmazás esetében a felhasználói felület, az API végpontok, a külső szolgáltatásokból érkező adatok és az adatbázis-műveletek egymásra épülnek, ezért egy kisebb módosítás is több rétegre lehet hatással. A tesztek szerepe ebben a folyamatban az, hogy ezekre a kockázatos kapcsolódási pontokra gyors visszajelzést adjanak.

A tesztelési stratégia elsősorban az MVP szempontjából kritikus funkciókra koncentrál. Ide tartozik a filmkeresés, a TMDB-ből érkező adatok feldolgozása, a watchlist kezelése, a filmek megnézettként való jelölése, az értékelés megadása, valamint az autentikációhoz kötött műveletek helyes kezelése. Ezek azok a részek, amelyek hibája közvetlenül rontaná a felhasználói élményt, vagy hibás adatállapotot eredményezne.

A tesztek több szinten jelennek meg a projektben. Az egyszerűbb unit tesztek az izolált függvényeket és service rétegeket ellenőrzik, például az API válaszok normalizálását vagy a formázó függvényeket. A komponens tesztek a felhasználói felület önálló elemeit vizsgálják, például az értékelő komponenst és a megnézettként jelöléshez tartozó dialógust. Az integrációs jellegű tesztek ezzel szemben már több réteg együttműködését ellenőrzik, például azt, hogy egy gombnyomás hatására a megfelelő API hívás történik-e, és a felület a válasz alapján frissül-e.

Fontos szempont volt, hogy a tesztek ne függjenek valódi külső szolgáltatásoktól. A TMDb API, a Supabase kliens, a böngésző `fetch` hívásai és bizonyos Next.js funkciók a tesztekben mockolt formában jelennek meg. Ez lehetővé teszi, hogy a tesztek gyorsan, determinisztikusan és internetkapcsolat vagy valós adatbázis nélkül fussanak le.

A tesztelés a CI/CD folyamatba is beépül. A GitHub Actions workflow külön lépésekben futtatja a lintelést, a TypeScript típusellenőrzést, a teszteket és a production buildet. Így a Pull Requestek ellenőrzése automatizált módon történhet, és a hibás vagy nem buildelhető módosítások még a fő ágba való bekerülés előtt kiszűrhetők.



7.1. ábra. Sikertelen CI pipeline a GitHub Actions felületén

7.2. Tesztkörnyezet és használt eszközök

A projekt tesztelése a Vitest-re épül. Ez jól illeszkedik a TypeScript és React alapú fejlesztési környezethez, valamint gyors visszajelzést ad a fejlesztés közben. A tesztek futtatása az `npm test` paranccsal történik, amely a `vitest run` parancsot indítja el. Fejlesztés közben használható a `test:watch` script is, amely folyamatosan figyeli a módosításokat, és automatikusan újrafuttatja az érintett teszteket.

A Vitest konfigurációja a `vitest.config.ts` fájlban található. A tesztek `jsdom` környezetben futnak, így a React komponensek olyan böngészőszerű környezetben vizsgálhatók, ahol elérhetők a DOM alapú műveletek. Ez különösen fontos az olyan komponenseknél, ahol a teszt nem csupán egy függvény visszatérési értékét ellenőrzi, hanem azt is, hogy a felhasználó milyen elemeket lát, és milyen interakciókat tud végrehajtani.

A komponensek teszteléséhez a projekt a React Testing Library eszközeit használja. Ennek előnye, hogy a tesztek a felhasználó szemszögéből vizsgálják a felületet: gombokra, szövegekre, linkekre és form elemekre keresnek rá, nem pedig a komponensek belső implementációjára. A felhasználói műveletek, például kattintások vagy mezők kitöltése, a `@testing-library/user-event` segítségével szimulálhatók.

A közös tesztbeállításokat a `vitest.setup.ts` fájl tartalmazza. Ebben kerül betöltésre a `jest-dom` kiegészítés, amely olvashatóbb állításokat biztosít a DOM elemek ellenőrzéséhez. Ugyanez a fájl minden teszt után elvégzi a `cleanup` műveletet is, így az egyes tesztek nem hagynak maguk után olyan állapotot, amely befolyásolná a következő teszt eredményét.

A külső függőségek és környezeti tényezők kezelésére a Vitest mockolási lehetőségei szolgálnak (a mock egy olyan helyettesítő objektum vagy függvény, amely a valódi működést utánozza a teszt során). A projektben például mockolt formában jelenik meg a `fetch`, a `TMDB` kliens, a `Supabase` kliens, valamint egyes `Next.js` funkciók. Ennek köszönhetően a tesztek nem igényelnek valódi hálózati kapcsolatot vagy adatbázis-hozzáférést, mégis ellenőrizni tudják, hogy az alkalmazás megfelelő kéréseket indít-e, és helyesen dolgozza-e fel a válaszokat.

7.3. Unit tesztek

A unit tesztek célja, hogy az alkalmazás kisebb, jól körülhatárolható részei önállóan is ellenőrizhetőek legyenek. Ezek a tesztek nem indítják el a teljes alkalmazást, nem használnak valódi adatbázist, és nem hívják meg ténylegesen a külső `TMDB` API-t. Ehelyett az adott függvény vagy service működését vizsgálják mockolt bemenetek és válaszok segítségével.

A `TMDB`-hez kapcsolódó tesztek a külső filmadatok lekérésének alapvető működését ellenőrzik. A `services/tmdb/client.test.ts` azt vizsgálja, hogy a kliens megfelelő URL-t épít-e fel, helye-

sen adja-e tovább a query paramétereket, és elküldi-e a bearer token alapú azonosítást. Emellett hibás TMDB válasz esetén azt is ellenőrzi, hogy a kliens értelmezhető hibát dob-e.

A `services/tmdb/search.test.ts` a filmkereséshez kapcsolódó adatnormalizálást teszteli. A TMDB válaszában szereplő mezők nem minden esetben egyeznek meg az alkalmazásban használt névkonvenciókkal, ezért a keresési eredményeket a service réteg átalakítja. A teszt azt ellenőrzi, hogy például a `poster_path`, `release_date` és `vote_average` mezők a frontend által használt formára alakulnak-e át.

A felhasználói listákhoz kapcsolódó unit tesztek a watched és watchlist service réteget fedik le. Ezek a tesztek azt ellenőrzik, hogy a service függvények a megfelelő API végpontokat hívják-e meg, megfelelő HTTP metódust használnak-e, és helyesen dolgozzák-e fel a sikeres vagy hibás válaszokat. Ilyen eset például egy film watchlisthez adása, eltávolítása, megnézettként jelölése, vagy egy API hiba saját hibatípusként való továbbítása.

Külön unit tesztek fedik le a megjelenítéshez kapcsolódó formázó függvényeket is. Ezek azt vizsgálják, hogy a szavazatszám, az átlagértékelés, a játékidő, a megjelenési dátum és a nyelv megfelelő formában jelenik-e meg a felhasználói felületen. Ezek a tesztek kisebbnek tűnnek, de fontosak, mert a film oldal és a keresési találatok több helyen is ezekre a formázott értékekre épülnek.

7.4. Komponens tesztek

A komponens tesztek az alkalmazás felhasználói felületének kisebb, önálló részeit tesztelik. Itt nem az a cél, hogy a komponens belső implementációja legyen ellenőrizve, hanem az, hogy a felhasználó szempontjából helyesen jelenik-e meg és használható-e az adott felületi elem. Emiatt a tesztek gombokra, szövegekre, linkekre, form mezőkre és látható állapotokra fókuszálnak rá.

Az egyik ilyen tesztelt komponens a `RatingInput`, amely a filmek személyes értékeléséhez kapcsolódik. A tesztek ellenőrzik, hogy megjelenik-e az öt értékelési lehetőség, a kiválasztott érték megfelelően aktív állapotba kerül-e, valamint kattintás után meghívódik-e a változást kezelő függvény.

A megnézettként jelöléshez tartozó dialógus szintén külön komponens tesztekkel rendelkezik. Ezek a tesztek azt vizsgálják, hogy értékelés megadása nélkül nem történik mentés, a kiválasztott értékelés és a megtekintési dátum helyesen kerül továbbításra, valamint mentés közben a felület nem

engedi a dialógus véletlen bezárását. Ez azért fontos, mert a `watched` funkció egyszerre kezel felhasználói interakciót, validációt és állapotváltozást.

Az autentikációhoz kötött műveleteknél az `AuthRequiredDialog` komponens jelenik meg, ha a felhasználó nincs bejelentkezve. A komponens tesztjei ellenőrzik, hogy a dialógus tartalmazza-e a megfelelő üzenetet, valamint a bejelentkezéshez és regisztrációhoz vezető linkeket. Ez biztosítja, hogy a felhasználó ne csak hibát kapjon, hanem egyértelmű továbblépési lehetőséget is.

A keresési találatok megjelenítését a `SearchResultCard` tesztjei fedik le. Ezek ellenőrzik, hogy a film címe, megjelenési éve, értékelése és a film oldalra mutató link helyesen jelenik-e meg. A tesztek a hiányos adatok kezelésére is kitérnek: poszter, leírás vagy megjelenési dátum hiányában a komponens fallback szövegeket jelenít meg, így a felület nem esik szét hiányos API válasz esetén sem.

Ezek a komponens tesztek azt biztosítják, hogy az alkalmazás fontosabb UI elemei önállóan is megbízhatóan működjenek. A tesztek felhasználói szemléletűek, ezért nem a komponensek belső állapotára épülnek, hanem arra, amit a felhasználó ténylegesen lát és használ.

7.5. CI/CD és automatikus ellenőrzések

A tesztek akkor nyújtanak valódi fejlesztési biztonságot, ha nem csak lokálisan, hanem a verziókezelési folyamatban is részt vesznek. A WatchDB projektben ezt a szerepet a GitHub Actions alapú CI workflow tölti be. A workflow a pull requestek létrehozásakor, valamint a `main` ágra történő push esetén indul el, így a módosítások automatikus ellenőrzésen mennek keresztül még azelőtt, hogy bekerülnének a fő ágba.

A CI folyamat több külön jobbura van bontva. A `Lint` job az ESLint ellenőrzést futtatja, és a kódstílushoz, valamint statikus kódelemzéshez kapcsolódó hibákat jelzi. A `Typecheck` job a TypeScript fordítóval ellenőrzi, hogy a projekt tartalmaz-e típus hibákat. Ez különösen fontos egy Next.js alkalmazásban, ahol a komponensek, service függvények, API route-ok és hookok több ponton is típusokra épülnek.

A `Test` job futtatja a Vitest alapú tesztkészletet. Ez magában foglalja a unit teszteket, a komponens teszteket, az integrációs jellegű teszteket és az API route teszteket is. Ennek célja, hogy a fontosabb üzleti logikák és felhasználói interakciók ne sérüljenek meg egy új módosítás hatására.

A negyedik ellenőrzési pont a Build job, amely a production buildet készíti el. Ez azért szükséges, mert előfordulhat, hogy a lokális fejlesztői környezetben egy módosítás működőképesnek tűnik, de a production build során mégis hibát okoz. A build job ezért azt ellenőrzi, hogy az alkalmazás telepíthető állapotban van-e.

Minden job saját környezetben fut: a workflow letölti a repository tartalmát, beállítja a Node.js környezetet, telepíti a függőségeket az `npm ci` paranccsal, majd lefuttatja az adott ellenőrzést. Ez tiszta és reprodukálható futási környezetet biztosít, mert a CI nem a fejlesztő gépének aktuális állapotára támaszkodik.

A "branch protection" beállítás segítségével a CI megakadályozza, hogy egy hibás kód kerüljön a fő ágba. Ilyenkor a GitHub csak akkor engedi a pull request mergelését, ha a kijelölt ellenőrzések sikeresen lefutottak. Így egy hibás lint eredmény, típushiba, sikertelen teszt vagy hibás build megakadályozhatja, hogy instabil kód kerüljön a fő ágba.

8. fejezet

Összefoglaló

8.1. Összefoglalás

A dolgozat a WatchDB példáján keresztül mutatta be, hogyan vezethet egy termékötletből strukturált tervezési és fejlesztési folyamat egy működő MVP-ig. A projekt során megjelent a követelmények meghatározása, a technológiai döntések indoklása, az implementáció, valamint a tesztelés és CI/CD szerepe. Az elkészült alkalmazás alapot biztosít a későbbi funkcionális és technikai bővítésekhez.

8.2. Továbbfejlesztési lehetőségek

A WatchDB jelenlegi változata az fő MVP funkciókra koncentrált, ezért a tovább fejlesztési lehetőségek megtalálhatóak az további MVP verziókban. Az egyik legfontosabb továbbfejlesztési lehetőség a közösségi funkciók bevezetése. Ide tartozhatnak a publikus profilok, más felhasználók követése, valamint egy aktivitási feed, ahol láthatóvá válik, hogy a felhasználó ismerősei milyen filmeket néztek meg vagy értékelték.

További fejlesztési irány lehet a személyes statisztikák és elemzések megjelenítése. A felhasználók számára hasznos lehet, ha áttekinthetik, milyen műfajú filmeket néznek leggyakrabban, hogyan változik az értékelési szokásuk, vagy egy adott időszakban mennyi filmet rögzítettek.

A filmek kezelését tovább bővíthetné az egyedi listák létrehozása. A jelenlegi watchlist és watched log mellett a felhasználók saját listákat is készíthetnének, például kedvenc filmekből, tematikus

válogatásokból vagy későbbi filmestekhez kapcsolódó gyűjteményekből. Ezek a listák később megoszthatóvá is válhatnának.

A keresési élmény is fejleszthető lenne. A jelenlegi cím alapú keresés mellett hasznos lehetne műfaj, év, népszerűség vagy értékelés szerinti szűrés és rendezés. Ez különösen akkor válna fontossá, ha az alkalmazás már nemcsak konkrét filmek megtalálására, hanem új filmek felfedezésére is szolgálna.

Technikai oldalról a tesztelési rendszer is tovább bővíthető. A jelenlegi Vitest alapú tesztek mellé később Playwright alapú end-to-end tesztek is készülhetnének, amelyek teljes felhasználói folyamatokat ellenőriznek böngészőben. Emellett hasznos lehetne coverage riport bevezetése, valamint külön tesztadatbázis használata az adatbázishoz kötött folyamatok megbízhatóbb ellenőrzésére.

Hosszabb távon a projekt mobilos használhatósága és teljesítménye is fejleszthető. Bár az alkalmazás webes felületként készült, a filmkövetés gyakran mobil eszközön történik, ezért a reszponzív felület finomítása, a betöltési sebesség javítása és akár PWA jellegű funkciók bevezetése is indokolt lehet. Ezek a fejlesztések hozzájárulhatnának ahhoz, hogy a WatchDB ne csak működő MVP, hanem hosszabb távon is kényelmesen használható filmkövető platform legyen.

Irodalomjegyzék

- [1] CERN, „World Wide Web at 35.” <https://home.cern/world-wide-web-35/>, 2024. Accessed: 2026-05-28.
- [2] Mozilla Developer Network, „Introduction to the server side.” https://developer.mozilla.org/en-US/docs/Learn_web_development/Extensions/Server-side/First_steps/Introduction. Accessed: 2026-05-28.
- [3] J. J. Garrett, „Ajax: A New Approach to Web Applications.” https://www.oceanpark.com/webmuseum/2005/garrett_on_ajax.html, 2005. Accessed: 2026-05-28.
- [4] Mozilla Developer Network, „Responsive web design.” https://developer.mozilla.org/en-US/docs/Learn_web_development/Core/CSS_layout/Responsive_Design. Accessed: 2026-05-28.
- [5] P. Mell and T. Grance, „The NIST Definition of Cloud Computing,” Tech. Rep. Special Publication 800-145, National Institute of Standards and Technology, 2011. <https://doi.org/10.6028/NIST.SP.800-145>. Accessed: 2026-05-28.
- [6] DevOps Research and Assessment, „Accelerate State of DevOps Report 2018.” <https://dora.dev/research/2018/dora-report/>, 2018. Accessed: 2026-05-28.
- [7] M. Fowler and J. Lewis, „Microservices.” <https://martinfowler.com/articles/microservices.html>, 2014. Accessed: 2026-05-28.
- [8] World Wide Web Consortium, „A Little History of the World Wide Web.” <https://www.w3.org/History.html>. Accessed: 2026-05-28.

- [9] Mozilla Developer Network, „What is a web server?.” https://developer.mozilla.org/en-US/docs/Learn_web_development/Howto/Web_mechanics/What_is_a_web_server. Accessed: 2026-05-28.
- [10] U.S. National Science Foundation, „Mosaic Launches an Internet Revolution.” <https://www.nsf.gov/news/mosaic-launches-internet-revolution>, 2004. Accessed: 2026-05-28.
- [11] World Wide Web Consortium, „The World Wide Web Consortium Issues HTML 4.0 as a W3C Recommendation.” <https://www.w3.org/press-releases/1997/html40-rec/>, 1997. Accessed: 2026-05-28.
- [12] World Wide Web Consortium, „HTML5.” <https://www.w3.org/TR/2014/REC-html5-20141028/>, 2014. Accessed: 2026-05-28.
- [13] D. Robinson and K. Coar, „The Common Gateway Interface (CGI) Version 1.1,” Tech. Rep. RFC 3875, RFC Editor, 2004. <https://doi.org/10.17487/RFC3875>. Accessed: 2026-05-28.
- [14] Mozilla Developer Network, „The web standards model.” https://developer.mozilla.org/en-US/docs/Learn_web_development/Getting_started/Web_standards/The_web_standards_model. Accessed: 2026-05-28.
- [15] Mozilla Developer Network, „Ajax.” <https://developer.mozilla.org/en-US/docs/Glossary/AJAX>. Accessed: 2026-05-28.
- [16] R. T. Fielding, M. Nottingham, and J. F. Reschke, „HTTP Semantics,” Tech. Rep. RFC 9110, RFC Editor, 2022. <https://doi.org/10.17487/RFC9110>. Accessed: 2026-05-28.
- [17] Mozilla Developer Network, „Introduction to client-side frameworks.” https://developer.mozilla.org/en-US/docs/Learn_web_development/Core/Frameworks_libraries/Introduction. Accessed: 2026-05-28.
- [18] Mozilla Developer Network, „SPA (Single-page application).” <https://developer.mozilla.org/en-US/docs/Glossary/SPA>. Accessed: 2026-05-28.

- [19] R. T. Fielding, *Architectural Styles and the Design of Network-based Software Architectures*. PhD thesis, University of California, Irvine, 2000. <https://ics.uci.edu/~fielding/pubs/dissertation/top.htm>. Accessed: 2026-05-28.
- [20] GraphQL Foundation, „GraphQL Specification, October 2021 Edition.” <https://spec.graphql.org/October2021/>, 2021. Accessed: 2026-05-28.
- [21] I. Fette and A. Melnikov, „The WebSocket Protocol,” Tech. Rep. RFC 6455, RFC Editor, 2011. <https://doi.org/10.17487/RFC6455>. Accessed: 2026-05-28.
- [22] World Wide Web Consortium, „Web Storage.” <https://www.w3.org/TR/2013/REC-webstorage-20130730/>, 2013. Accessed: 2026-05-28.
- [23] Mozilla Developer Network, „Offline and background operation.” https://developer.mozilla.org/en-US/docs/Web/Progressive_web_apps/Guides/Offline_and_background_operation. Accessed: 2026-05-28.
- [24] J. Wagner, „Client-side rendering of HTML and interactivity.” <https://web.dev/articles/client-side-rendering-of-html-and-interactivity>. Accessed: 2026-05-28.
- [25] World Wide Web Consortium, „Mobile Web Best Practices 1.0.” <https://www.w3.org/TR/2008/REC-mobile-bp-20080729/>, 2008. Accessed: 2026-05-28.
- [26] E. Marcotte, „Responsive Web Design.” <https://alistapart.com/article/responsive-web-design/>, 2010. Accessed: 2026-05-28.
- [27] World Wide Web Consortium, „Media Queries.” <https://www.w3.org/TR/2012/REC-css3-mediaqueries-20120619/>, 2012. Accessed: 2026-05-28.
- [28] World Wide Web Consortium, „Touch Events.” <https://www.w3.org/TR/2013/REC-touch-events-20131010/>, 2013. Accessed: 2026-05-28.
- [29] L. Wroblewski, *Mobile First*. A Book Apart, 2011. https://www.lukew.com/resources/mobile_first.asp. Accessed: 2026-05-28.

- [30] A. Russell, „Progressive Web Apps: Escaping Tabs Without Losing Our Soul.” <https://medium.com/@slightlylate/progressive-apps-escaping-tabs-without-losing-our-soul-3b93a8561955>, 2015. Accessed: 2026-05-28.
- [31] Mozilla Developer Network, „What is a progressive web app?.” https://developer.mozilla.org/en-US/docs/Web/Progressive_web_apps/Guides/What_is_a_progressive_web_app. Accessed: 2026-05-28.
- [32] Mozilla Developer Network, „MediaDevices: getUserMedia() method.” <https://developer.mozilla.org/en-US/docs/Web/API/MediaDevices/getUserMedia>. Accessed: 2026-05-28.
- [33] World Wide Web Consortium, „Geolocation API.” <https://www.w3.org/TR/2024/REC-geolocation-20240613/>, 2024. Accessed: 2026-05-28.
- [34] Amazon Web Services, „Introducing AWS Lambda.” <https://aws.amazon.com/about-aws/whats-new/2014/11/13/introducing-aws-lambda/>, 2014. Accessed: 2026-05-28.
- [35] HTTP Archive, „Security: 2025 Web Almanac.” <https://almanac.httparchive.org/en/2025/security>, 2026. Accessed: 2026-05-28.
- [36] Let’s Encrypt, „Let’s Encrypt Launch Schedule.” <https://letsencrypt.org/2015/06/16/lets-encrypt-launch-schedule/>, 2015. Accessed: 2026-05-28.
- [37] E. Schechter, „A milestone for Chrome security: marking HTTP as "not secure".” <https://blog.google/products/chrome/milestone-chrome-security-marking-http-not-secure/>, 2018. Accessed: 2026-05-28.
- [38] Cloudflare, „What is a CDN edge server?.” <https://www.cloudflare.com/learning/cdn/glossary/edge-server/>. Accessed: 2026-05-28.
- [39] K. Beck, M. Beedle, A. van Bennekum, A. Cockburn, W. Cunningham, M. Fowler, J. Grenning, J. Highsmith, A. Hunt, R. Jeffries, J. Kern, B. Marick, R. C. Martin, S. Mellor, K. Schwaber, J. Sutherland, and D. Thomas, „Manifesto for Agile Software Development.” <https://agilemanifesto.org/>, 2001. Accessed: 2026-05-28.

- [40] World Wide Web Consortium and Web Hypertext Application Technology Working Group, „Memorandum of Understanding Between W3C and WHATWG.” <https://www.w3.org/2019/04/WHATWG-W3C-MOU.html>, 2019. Accessed: 2026-05-28.
- [41] Ecma International, „ECMAScript 2024 Language Specification.” <https://262.ecma-international.org/15.0/>, 2024. Accessed: 2026-05-28.
- [42] World Wide Web Consortium, „WebAssembly becomes a W3C Recommendation.” <https://www.w3.org/press-releases/2019/wasm/>, 2019. Accessed: 2026-05-28.
- [43] React Native Team, „Introduction.” <https://reactnative.dev/docs/getting-started>. Accessed: 2026-05-28.
- [44] Flutter Team, „Build apps for any screen.” <https://flutter.dev/>. Accessed: 2026-05-28.
- [45] Encyclopaedia Britannica, „ARPANET.” <https://www.britannica.com/topic/ARPANET>, 2026. Accessed: 2026-05-28.
- [46] Computer History Museum, „Internet History 1962 to 1992.” <https://www.computerhistory.org/internethistory/>. Accessed: 2026-05-28.
- [47] HTTP Archive, „Generative AI: 2025 Web Almanac.” <https://almanac.httparchive.org/en/2025/generative-ai>, 2026. Accessed: 2026-05-28.
- [48] TechTarget, „What is a Web Application?.” <https://www.techtarget.com/searchsoftwarequality/definition/Web-application-Web-app>. Accessed: 2026-06-01.
- [49] Mozilla Developer Network, „Client-Server Overview.” https://developer.mozilla.org/en-US/docs/Learn_web_development/Extensions/Server-side/First_steps/Client-Server_overview. Accessed: 2026-06-01.
- [50] web.dev, „Architecture.” <https://web.dev/learn/pwa/architecture>, 2022. Accessed: 2026-06-01.
- [51] A. Osmani and J. Miller, „Rendering on the Web.” <https://web.dev/articles/rendering-on-the-web>, 2019. Accessed: 2026-06-01.

- [52] IBM, „What is the software development life cycle (SDLC)?.” <https://www.ibm.com/think/topics/sdlc>. Accessed: 2026-06-01.
- [53] IBM, „Agile vs. Waterfall: What’s the Difference?”. <https://www.ibm.com/think/topics/agile-vs-waterfall>. Accessed: 2026-06-01.
- [54] K. Schwaber and J. Sutherland, „The Scrum Guide.” <https://scrumguides.org/scrum-guide.html>, 2020. Accessed: 2026-06-01.
- [55] Kanban Guides, „The Kanban Guide.” <https://kanbanguides.org/english/>, 2025. Accessed: 2026-06-01.
- [56] Atlassian, „What is DevOps?”. <https://www.atlassian.com/devops>. Accessed: 2026-06-01.
- [57] Amazon Web Services, „What is CI/CD?”. <https://aws.amazon.com/what-is/ci-cd/>. Accessed: 2026-06-01.
- [58] D. Norman and J. Nielsen, „The Definition of User Experience (UX).” <https://www.nngroup.com/articles/definition-user-experience/>, 1998. Accessed: 2026-06-01.
- [59] World Wide Web Consortium, „Introduction to Web Accessibility.” <https://www.w3.org/WAI/fundamentals/accessibility-intro/>. Accessed: 2026-06-01.
- [60] Amazon Web Services, „What is Full Stack Development?”. <https://aws.amazon.com/what-is/full-stack-development/>. Accessed: 2026-06-01.
- [61] IBM, „What is Shift-left Testing?”. <https://www.ibm.com/think/topics/shift-left-testing>. Accessed: 2026-06-01.
- [62] International Software Testing Qualifications Board, „ISTQB Glossary: Regression Testing.” https://glossary.istqb.org/en_US/term/regression-testing. Accessed: 2026-06-01.
- [63] International Software Testing Qualifications Board, „ISTQB Glossary: Test Automation.” https://glossary.istqb.org/en_US/term/test-automation. Accessed: 2026-06-01.

- [64] IEEE Standards Association, „IEEE/ISO/IEC 29148-2018: Systems and software engineering – Life cycle processes – Requirements engineering.” <https://standards.ieee.org/ieee/29148/6937/>, 2018. Accessed: 2026-06-01.
- [65] International Organization for Standardization, „ISO/IEC 25010:2023: Systems and software Quality Requirements and Evaluation (SQuaRE) – Product quality model.” <https://www.iso.org/standard/78176.html>, 2023. Accessed: 2026-06-01.
- [66] International Organization for Standardization, „ISO/IEC/IEEE 42010:2022: Software, systems and enterprise – Architecture description.” <https://www.iso.org/standard/74393.html>, 2022. Accessed: 2026-06-01.
- [67] Mozilla Developer Network, „Introduction to automated testing.” https://developer.mozilla.org/en-US/docs/Learn_web_development/Extensions/Testing/Automated_testing. Accessed: 2026-06-01.
- [68] IEEE Standards Association, „IEEE/ISO/IEC 14764-2021: Software engineering – Software life cycle processes – Maintenance.” <https://standards.ieee.org/ieee/14764/7701/>, 2021. Accessed: 2026-06-01.
- [69] M. Souppaya, K. Scarfone, and D. Dodson, „Secure Software Development Framework (SSDF) Version 1.1: Recommendations for Mitigating the Risk of Software Vulnerabilities,” Tech. Rep. Special Publication 800-218, National Institute of Standards and Technology, 2022. <https://doi.org/10.6028/NIST.SP.800-218>. Accessed: 2026-06-01.
- [70] Vercel, „Next.js Docs: App Router.” <https://nextjs.org/docs/app>. Accessed: 2026-06-01.
- [71] Meta Open Source, „React Docs: Describing the UI.” <https://react.dev/learn/describing-the-ui>. Accessed: 2026-06-01.
- [72] Supabase, „Creating a Supabase client for SSR.” <https://supabase.com/docs/guides/auth/server-side/nextjs>. Accessed: 2026-06-01.

- [73] The Movie Database, „TMDB API: Getting Started.” <https://developer.themoviedb.org/docs/getting-started>. Accessed: 2026-06-01.
- [74] TanStack, „TanStack Query: Overview.” <https://tanstack.com/query/latest/docs/framework/react/overview>. Accessed: 2026-06-01.
- [75] Tailwind Labs, „Tailwind CSS: Styling with utility classes.” <https://tailwindcss.com/docs/styling-with-utility-classes>. Accessed: 2026-06-01.
- [76] Radix UI, „Radix Primitives: Introduction.” <https://www.radix-ui.com/primitives/docs/overview/introduction>. Accessed: 2026-06-01.
- [77] shadcn, „shadcn/ui: Components.” <https://ui.shadcn.com/docs/components>. Accessed: 2026-06-01.
- [78] IMDb, „Watchlist FAQ.” <https://help.imdb.com/article/imdb/track-movies-tv/watchlist-faq/G9PA556494DM8YBA>. Accessed: 2026-06-07.
- [79] Trakt, „Discover. Track. Share.” <https://widgets.trakt.tv/>. Accessed: 2026-06-07.
- [80] Letterboxd, „Letterboxd: Social film discovery.” <https://letterboxd.com/>. Accessed: 2026-06-07.